

iTeam University

Tunis - Tunisia



Projet Semestriel (S4)

Dans le cadre du : **Ingénieur en Cybersécurité**

Thème :

**Conception, déploiement et analyse de sécurité d'une application
web hébergée sur AWS via la conteneurisation Docker**

Élaboré par : Mohamed Salem KHYARHOUM

Encadré par : M. Raihane Modhaffer

Année Universitaire : 2025-2026

Remerciements

Au terme de ce projet semestriel, je tiens à exprimer ma profonde reconnaissance à toutes les personnes qui, par leur soutien, leurs conseils ou leur présence, ont contribué à la réalisation de ce travail.

Je remercie tout d'abord **Allah le Tout-Puissant** de m'avoir donné la santé, la patience et la volonté nécessaires pour mener à bien ce projet.

Mes plus sincères remerciements s'adressent à mon encadrant, **M. Raihane Modhaffer**, pour sa disponibilité, ses conseils éclairés et son suivi rigoureux tout au long de la réalisation de ce projet. Ses remarques pertinentes m'ont permis d'orienter mon travail dans la bonne direction et d'en améliorer la qualité.

Je tiens également à remercier l'ensemble du corps enseignant de **iTeam University** pour la qualité de la formation reçue durant ce semestre, ainsi que pour l'environnement académique stimulant qu'ils ont su créer.

Mes remerciements vont aussi aux **membres du jury** qui ont accepté d'évaluer ce travail. Leurs critiques constructives constitueront une précieuse contribution à mon développement professionnel futur.

Enfin, j'adresse une pensée toute particulière à ma **famille** et à mes **amis** pour leur soutien indéfectible, leur patience et leurs encouragements, qui m'ont permis de surmonter les moments difficiles et d'aller jusqu'au bout de ce projet.

À toutes et à tous, merci.

Mohamed Salem KHYARHOUM

Résumé

Ce projet de fin d'études porte sur la conception, le développement, le déploiement et l'analyse de sécurité d'une application web de prise de notes personnelle, baptisée **WebCyber**. L'application repose sur le *framework* Python **Flask** pour la couche métier, sur **PostgreSQL** pour la persistance des données, et sur **Nginx** comme proxy inverse assurant la terminaison TLS. L'ensemble est **conteneurisé** avec Docker et orchestré via Docker Compose, puis **déployé sur AWS** (instance EC2 t2.micro sous Ubuntu Server 22.04 LTS), avec une résolution de nom de domaine gérée par **Route 53** et un certificat HTTPS automatique fourni par **Let's Encrypt**.

Une attention particulière a été portée à la sécurité, organisée en **six couches** : pare-feu réseau (*Security Group* AWS), durcissement du système, isolation Docker, configuration TLS et en-têtes de sécurité Nginx, contrôles applicatifs (authentification, hachage PBKDF2-SHA256, protection CSRF, validation des entrées, isolation des données par utilisateur) et sécurité de la base de données. La solution déployée a ensuite été soumise à une campagne d'évaluation à l'aide de **Nmap**, **Nikto** et **testssl.sh**, dont les résultats sont analysés et discutés.

Abstract

This end-of-studies project covers the design, development, deployment and security assessment of a personal note-taking web application named **WebCyber**. The application is built on the Python **Flask** framework for business logic, **PostgreSQL** for data persistence, and **Nginx** as a reverse proxy handling TLS termination. The whole stack is **containerised** with Docker and orchestrated through Docker Compose, then **deployed on AWS** (EC2 t2.micro instance running Ubuntu Server 22.04 LTS), with DNS resolution provided by **Route 53** and HTTPS certificates automatically issued by **Let's Encrypt**.

Security has been treated as a first-class concern and structured into **six layers**: AWS Security Group network firewall, host hardening, Docker isolation, Nginx TLS and security headers, application-level controls (authentication, PBKDF2-SHA256 password hashing, CSRF protection, input validation, per-user data isolation) and database security. The deployed solution was then assessed using **Nmap**, **Nikto** and **testssl.sh**, and the results are analysed and discussed.

Table des matières

Introduction Générale	11
1 Contexte	11
2 Problématique	11
3 Objectifs	11
4 Approche méthodologique	12
5 Plan du rapport	12
1 CONTEXTE GÉNÉRAL ET ÉTAT DE L'ART	14
1. 1 Introduction	15
1. 2 Cloud computing	15
1. 2.1 Définition	15
1. 2.2 Modèles de service	15
1. 2.3 Amazon Web Services (AWS)	15
1. 3 Conteneurisation	15
1. 3.1 Docker	15
1. 3.2 Docker Compose	15
1. 3.3 Bénéfices	16
1. 4 Architecture des applications web modernes	16
1. 4.1 Modèle 3-tiers	16
1. 4.2 Proxy inverse (Nginx)	16
1. 5 Cybersécurité applicative	16
1. 5.1 Top 10 OWASP	16
1. 5.2 Défense en profondeur	16
1. 5.3 TLS/HTTPS	17
1. 6 Conclusion	17
2 ANALYSE ET SPÉCIFICATION DES BESOINS	18
2. 1 Introduction	19
2. 2 Acteurs du système	19
2. 3 Besoins fonctionnels	19
2. 3.1 Tableau des besoins	19
2. 3.2 Tableau des besoins	19
2. 3.3 Diagramme de cas d'utilisation	20
2. 3.4 User Stories (approche agile)	20
2. 3.5 Product Backlog	21
2. 4 Besoins non fonctionnels	22
2. 4.1 Sécurité	22
2. 4.2 Performance et disponibilité	22
2. 4.3 Déploiement et maintenabilité	22
2. 4.4 Ergonomie	22
2. 4.5 Périmètre exclu	23
2. 5 Synthèse des exigences	23
2. 6 Conclusion du chapitre	23
3 CONCEPTION	24
3. 1 Introduction	25

3. 2	Architecture globale	25
3. 2.1	Limites de confiance (trust boundaries)	25
3. 2.2	Réduction de la surface d'attaque	26
3. 2.3	Composants et responsabilités	26
3. 3	Infrastructure AWS	26
3. 3.1	Choix de la région	26
3. 3.2	Composants AWS retenus	26
3. 3.3	Règles du Security Group	27
3. 4	Architecture Docker	28
3. 4.1	Vue d'ensemble	28
3. 4.2	Spécifications des conteneurs	29
3. 4.3	Justification des choix techniques	29
3. 5	Modèle de données	29
3. 5.1	Description des entités	30
3. 5.2	Considérations de sécurité	30
3. 6	Diagrammes de séquence	31
3. 6.1	Authentification	31
3. 6.2	Création d'une note	32
3. 7	Conception de la sécurité - défense en profondeur	33
3. 7.1	Couche 1 - Security Group (AWS)	33
3. 7.2	Couche 2 - Système hôte (EC2)	33
3. 7.3	Couche 3 - Isolation Docker	34
3. 7.4	Couche 4 - Nginx + TLS	34
3. 7.5	Couche 5 - Application Flask	34
3. 7.6	Couche 6 - Base de données	35
3. 8	Récapitulatif de la conception	35
3. 9	Conclusion du chapitre	36
4	RÉALISATION ET DÉPLOIEMENT	37
4. 1	Introduction	38
4. 2	Vue d'ensemble de l'infrastructure	38
4. 3	Stack technologique	38
4. 4	Infrastructure as Code avec Terraform	39
4. 4.1	Philosophie	39
4. 4.2	Ressources provisionnées	39
4. 5	Conteneurisation et orchestration	40
4. 5.1	Philosophie	40
4. 5.2	Structure des conteneurs	40
4. 5.3	Points de sécurité sur les conteneurs	40
4. 5.4	Réseau : pourquoi seul Nginx est exposé ?	40
4. 6	Déploiement continu (CI/CD)	41
4. 6.1	Philosophie	41
4. 6.2	Architecture du pipeline	41
4. 6.3	Gestion des secrets	41
4. 7	Configuration DNS et HTTPS	42
4. 7.1	DNS avec Route 53	42
4. 7.2	TLS avec Let's Encrypt	42

4. 7.3	Configuration Nginx	42
4. 8	Flux d'une requête HTTPS	43
4. 9	Vérification post-déploiement	43
4. 10	Interfaces de l'application	43
4. 11	Conclusion du chapitre	44
5	TESTS ET ANALYSE DE SÉCURITÉ	45
5. 1	Introduction	46
5. 2	Stratégie de test	46
5. 3	Tests fonctionnels	46
5. 3.1	Inscription et authentification	46
5. 3.2	Gestion des notes (CRUD)	47
5. 3.3	Test critique : isolation des données	47
5. 4	Analyse statique de l'IaC avec tfsec	48
5. 4.1	Principe et positionnement	48
5. 4.2	Résultats et décisions	48
5. 4.3	Interprétation des décisions	48
5. 4.4	Intégration CI/CD	48
5. 5	Vérification de l'infrastructure déployée	49
5. 5.1	Résolution DNS	49
5. 5.2	État des conteneurs	49
5. 5.3	Connectivité applicative	49
5. 6	Scan de ports avec Nmap	49
5. 6.1	Objectif et méthode	49
5. 6.2	Résultats et interprétation	50
5. 6.3	Conclusion et recommandations	50
5. 7	Scan applicatif Web avec Nikto	50
5. 7.1	Objectif et méthode	50
5. 7.2	Résultats et interprétation	51
5. 7.3	Conclusion	51
5. 8	Évaluation TLS	51
5. 8.1	Objectif et méthode	51
5. 8.2	Résultats et interprétation	52
5. 8.3	Conclusion	52
5. 9	Synthèse des audits	53
5. 10	Limites de la campagne	53
5. 11	Conclusion du chapitre	54
6	CONCLUSION GÉNÉRALE	55
6. 1	Bilan du projet	56
6. 2	Apports méthodologiques	56
6. 2.1	Retour sur la démarche agile	56
6. 3	Limites et perspectives d'évolution	57
6. 4	Conclusion	57
	Bibliographie	58

Liste des figures

Fig. 1	Diagramme de cas d'utilisation de WebCyber.	20
Fig. 2	Architecture globale - vue d'ensemble.	25
Fig. 3	Infrastructure AWS détaillée.	27
Fig. 4	Architecture Docker - trois conteneurs sur réseau bridge.	28
Fig. 5	Diagramme de classes - modèle de données.	29
Fig. 6	Diagramme de séquence - authentification.	31
Fig. 7	Diagramme de séquence - création d'une note.	32
Fig. 8	Six couches de sécurité (defense-in-depth).	33
Fig. 9	Diagramme de déploiement UML - artefacts physiques et logiques.	38
Fig. 10	Flux d'une requête HTTPS, des en-têtes au stockage.	43
Fig. 11	Interface de connexion / inscription (Tailwind CSS, responsive).	44
Fig. 12	Interface de gestion des notes (liste, création, édition, archive, corbeille).	44

Liste des tableaux

Tableau 1	Organisation des sprints du projet WebCyber.	12
Tableau 2	Tableau des besoins fonctionnels.	19
Tableau 3	Tableau des besoins fonctionnels.	20
Tableau 4	User Stories du projet WebCyber.	20
Tableau 5	User Stories du projet WebCyber.	21
Tableau 6	Product Backlog du projet WebCyber.	21
Tableau 7	Exigences de sécurité.	22
Tableau 8	Exigences de performance et disponibilité.	22
Tableau 9	Exigences de déploiement et maintenabilité.	22
Tableau 10	Synthèse des exigences du projet.	23
Tableau 11	Composants et responsabilités.	26
Tableau 12	Règles entrantes du Security Group webcyber-sg.	27
Tableau 13	Spécifications des conteneurs.	29
Tableau 14	Configuration du Security Group.	33
Tableau 15	Configuration du système hôte.	34
Tableau 16	Mesures d'isolation Docker.	34
Tableau 17	Configuration Nginx et TLS.	34
Tableau 18	Mesures de sécurité de l'application Flask.	35
Tableau 19	Mesures de sécurité de la base de données.	35
Tableau 20	Synthèse de l'architecture technique.	35
Tableau 21	Synthèse de la pile technologique.	39
Tableau 22	Ressources Terraform provisionnées.	39
Tableau 23	Spécifications des conteneurs.	40
Tableau 24	Jobs du pipeline GitHub Actions.	41
Tableau 25	Secrets configurés dans GitHub Actions.	41
Tableau 26	Enregistrements DNS de la zone hébergée Route 53.	42
Tableau 27	En-têtes de sécurité HTTP injectés par Nginx.	42
Tableau 28	Phases de la campagne de validation.	46
Tableau 29	Tests d'inscription.	46
Tableau 30	Tests d'authentification.	47
Tableau 31	Tests CRUD sur les notes.	47
Tableau 32	Tests d'isolation par utilisateur.	47
Tableau 33	Constats tfsec sur le module Terraform et décisions associées.	48
Tableau 34	Résultats du scan Nmap.	50
Tableau 35	Synthèse des résultats Nikto.	51
Tableau 36	Synthèse de l'évaluation TLS.	52
Tableau 37	Tableau de bord de sécurité.	53

Liste des acronymes

Acronyme	Signification
ACME	<i>Automatic Certificate Management Environment (protocole Let's Encrypt)</i>
AMI	<i>Amazon Machine Image</i>
API	<i>Application Programming Interface</i>
AWS	<i>Amazon Web Services</i>
CI/CD	<i>Continuous Integration / Continuous Deployment</i>
CIDR	<i>Classless Inter-Domain Routing</i>
CRUD	<i>Create, Read, Update, Delete</i>
CSP	<i>Content Security Policy</i>
CSRF	<i>Cross-Site Request Forgery</i>
DNS	<i>Domain Name System</i>
EBS	<i>Elastic Block Store</i>
EC2	<i>Elastic Compute Cloud</i>
EIP	<i>Elastic IP Address</i>
GHA	<i>GitHub Actions</i>
GHCR	<i>GitHub Container Registry</i>
HSTS	<i>HTTP Strict Transport Security</i>
HTTP	<i>Hypertext Transfer Protocol</i>
HTTPS	<i>HTTP Secure</i>
IaaS	<i>Infrastructure as a Service</i>
IaC	<i>Infrastructure as Code</i>
IGW	<i>Internet Gateway</i>
IDOR	<i>Insecure Direct Object Reference</i>
IMDSv2	<i>Instance Metadata Service version 2</i>
IP	<i>Internet Protocol</i>
LTS	<i>Long-Term Support</i>
MVC	<i>Modèle-Vue-Contrôleur</i>
NIST	<i>National Institute of Standards and Technology</i>
ORM	<i>Object-Relational Mapping</i>
OWASP	<i>Open Web Application Security Project</i>
PaaS	<i>Platform as a Service</i>
PBKDF2	<i>Password-Based Key Derivation Function 2</i>
PFE	<i>Projet de Fin d'Études</i>
SaaS	<i>Software as a Service</i>
SAST	<i>Static Application Security Testing</i>

SG	<i>Security Group</i>
SQL	<i>Structured Query Language</i>
SSH	<i>Secure Shell</i>
SSL	<i>Secure Sockets Layer</i>
TCP	<i>Transmission Control Protocol</i>
TLD	<i>Top-Level Domain</i>
TLS	<i>Transport Layer Security</i>
URL	<i>Uniform Resource Locator</i>
UML	<i>Unified Modeling Language</i>
VPC	<i>Virtual Private Cloud</i>
WSGI	<i>Web Server Gateway Interface</i>
XSS	<i>Cross-Site Scripting</i>

Introduction Générale

1 Contexte

À l'ère du numérique, la plupart des services quotidiens — banque, communication, commerce, administration — reposent sur des applications web hébergées dans le **cloud**. Cette transformation s'accompagne d'une hausse constante des cyberattaques.

> Selon le **Verizon Data Breach Investigations Report 2023**, plus de 80% des compromissions exploitent des vulnérabilités applicatives ou des erreurs de configuration.

Développer une application web ne se limite donc plus à écrire du code fonctionnel. Il faut aussi maîtriser l'hébergement, le réseau, le cycle de déploiement et — surtout — la **surface d'attaque**.

Les pratiques **DevOps** et la conteneurisation (Docker) ont profondément changé le paysage technique. Docker offre une portabilité parfaite entre les environnements. Les fournisseurs cloud comme AWS permettent de provisionner des infrastructures complexes en quelques minutes.

Mais la maîtrise isolée de ces outils ne suffit pas. Leur intégration sécurisée reste un défi, même pour des applications simples.

2 Problématique

Combien de bases de données sont exposées par erreur sur Internet ? Combien de certificats TLS expirés, d'APIs non authentifiées, de conteneurs tournant en root, de pare-feu mal configurés ?

La difficulté n'est pas technique — les solutions existent. Elle est **méthodologique** : comment articuler ces bonnes pratiques de manière cohérente, vérifiable et documentée ?

Comment concevoir, déployer et valider la sécurité d'une application web cloud-native, en appliquant systématiquement les bonnes pratiques — depuis la couche réseau jusqu'au code applicatif — dans un projet académique individuel ?

Ce projet apporte une réponse concrète à cette question.

3 Objectifs

Le projet **WebCyber** poursuit six objectifs principaux :

1. **Concevoir** une application réaliste (prise de notes, authentification, CRUD) et la conteneuriser intégralement avec Docker Compose.

2. **Déployer** sur une infrastructure cloud minimale mais professionnelle (AWS EC2, Security Group, Elastic IP, Route 53).
3. **Sécuriser en profondeur** : réseau, système hôte, conteneurs, proxy Nginx (TLS), application Flask (hachage, CSRF, isolation) et base de données.
4. **Automatiser** le déploiement via CI/CD (GitHub Actions) avec une gate de sécurité (tfsec).
5. **Auditer** la sécurité déployée avec des outils standards (Nmap, Nikto, testssl.sh).
6. **Documenter** l'ensemble pour permettre la reproductibilité par un tiers.

4 Approche méthodologique

Le projet suit une **démarche agile adaptée au contexte académique individuel**. Trois pratiques agiles ont été retenues :

- **Sprints de 2-3 semaines** : chaque sprint livre un incrément tangible (chapitre, composant, déploiement).
- **Revue régulières** : points hebdomadaires avec l'encadrant pour ajuster les priorités.
- **Kanban personnel** : suivi visuel des tâches (À faire → En cours → Terminé).

La planification s'organise en cinq sprints :

Sprint	Durée	Phase	Objectifs	Livrables
1	2 sem.	Analyse	État de l'art, cadrage	Chapitres 1-2
2	3 sem.	Conception	Architecture, UML	Chapitre 3
3	3 sem.	Réalisation	Développement, conteneurisation	Chapitre 4
4	2 sem.	Déploiement	AWS, HTTPS, CI/CD	Chapitre 4
5	2 sem.	Validation	Tests, audit sécurité	Chapitre 5

Tableau 1. – Organisation des sprints du projet WebCyber.

Cette organisation garantit une progression mesurable, des revues régulières et des ajustements en continu.

5 Plan du rapport

Le mémoire s'articule autour de cinq chapitres :

- **Chapitre 1 — Contexte et état de l'art** : cloud computing, conteneurisation, architecture web, OWASP Top 10.
- **Chapitre 2 — Analyse des besoins** : acteurs, cas d'utilisation, besoins fonctionnels et non fonctionnels, user stories.
- **Chapitre 3 — Conception** : architecture globale (AWS + Docker), modèle de données, sécurité en profondeur (6 couches).

- **Chapitre 4 — Réalisation et déploiement** : infrastructure as code (Terraform), CI/CD (GitHub Actions), HTTPS (Let's Encrypt).
- **Chapitre 5 — Tests et audit de sécurité** : scans réseau, analyse applicative, évaluation TLS, synthèse des résultats.

Une **conclusion générale** dresse le bilan et propose des perspectives (Kubernetes, CI/CD complet, monitoring).

Chapitre 1

1 CONTEXTE GÉNÉRAL ET ÉTAT DE L'ART

1. 1 Introduction

Ce premier chapitre présente les concepts technologiques utilisés dans ce projet semestriel : *cloud computing*, conteneurisation, architecture des applications web modernes et principes de cybersécurité. Ces bases conceptuelles permettent de motiver les choix de conception et d'implémentation dans les chapitres suivants.

1. 2 Cloud computing

1. 2.1 Définition

Le **cloud computing** désigne la mise à disposition à la demande, via Internet, de ressources informatiques (calcul, stockage, réseau, bases de données). Cette technologie est utilisée dans ce projet pour héberger l'application sur AWS.

1. 2.2 Modèles de service

Trois modèles de service principaux sont distingués :

- **IaaS** (*Infrastructure as a Service*) : ressources virtualisées brutes (machines, réseaux, disques);
- **PaaS** (*Platform as a Service*) : infrastructure + environnement d'exécution;
- **SaaS** (*Software as a Service*) : application complète.

Pour ce projet, le modèle **IaaS** (AWS EC2) est retenu pour sa flexibilité pédagogique.

1. 2.3 Amazon Web Services (AWS)

AWS est le leader mondial du *cloud* public. Les services utilisés dans ce projet sont :

- **EC2** : instances de machines virtuelles;
- **Elastic IP** : adresse IPv4 publique fixe;
- **Security Group** : pare-feu *stateful*;
- **Route 53** : service DNS.

Le choix d'AWS se justifie par sa documentation complète et son *free tier* adapté aux projets académiques.

1. 3 Conteneurisation

1. 3.1 Docker

Docker est l'outil standard pour la conteneurisation. Deux concepts clés :

- l'**image** : un instantané immuable;
- le **conteneur** : une instance d'exécution.

1. 3.2 Docker Compose

Docker Compose permet de décrire une pile applicative complète (services, réseaux, volumes) dans un fichier `docker-compose.yml`.

1. 3.3 Bénéfices

- **Reproductibilité** : mêmes environnements partout;
- **Isolation** : chaque service (Nginx, Flask, PostgreSQL) tourne indépendamment;
- **Sécurité** : réseau interne exposant uniquement Nginx;
- **Portabilité** : déployable sur toute machine supportant Docker.

1. 4 Architecture des applications web modernes

1. 4.1 Modèle 3-tiers

Une application web suit un découpage en trois couches :

- **Présentation** : navigateur client;
- **Logique métier** : serveur applicatif;
- **Données** : base de données.

1. 4.2 Proxy inverse (Nginx)

Un **proxy inverse** se place entre les clients et l'application. Ses fonctions :

- **Terminaison TLS** : gestion HTTPS;
- **Routage** : distribution des requêtes;
- **Cache** et compression;
- **Sécurité** : en-têtes HTTP.

Nginx est choisi pour sa légèreté et sa robustesse.

1. 5 Cybersécurité applicative

1. 5.1 Top 10 OWASP

L'OWASP publie la liste des dix risques applicatifs les plus critiques :

1. **Broken Access Control** : contournement des contrôles d'accès;
2. **Cryptographic Failures** : chiffrement inadéquat;
3. **Injection** : SQL, NoSQL;
4. **Security Misconfiguration** : configurations par défaut;
5. **Vulnerable Components** : bibliothèques non maintenues.

1. 5.2 Défense en profondeur

La **défense en profondeur** empile plusieurs barrières de sécurité. Dans ce projet, six couches sont implémentées :

1. Pare-feu AWS Security Group
2. Certificat TLS (Let's Encrypt)
3. Proxy inverse Nginx durci
4. Application Flask sécurisée
5. Conteneurs isolés

6. Base de données non exposée

1. 5.3 TLS/HTTPS

TLS chiffre les communications. **HTTPS** = HTTP + TLS. **Let's Encrypt** délivre des certificats gratuits valides 90 jours.

1. 6 Conclusion

Ce chapitre a présenté les concepts clés : cloud AWS, conteneurs Docker, architecture web et sécurité. Le chapitre suivant formalise les besoins fonctionnels de l'application WebCyber.

Chapitre 2

2 ANALYSE ET SPÉCIFICATION DES BESOINS

2. 1 Introduction

Ce chapitre formalise ce que l'application **WebCyber** doit accomplir et dans quelles conditions. Il présente successivement :

- les acteurs du système ;
- les besoins fonctionnels (cas d'utilisation, user stories) ;
- le **Product Backlog** agile ;
- les besoins non fonctionnels (sécurité, performance, déploiement).

Cette analyse servira de cahier des charges pour les chapitres de conception et de réalisation.

2. 2 Acteurs du système

Deux acteurs interagissent avec l'application :

Visiteur : Tout internaute non authentifié. Il peut accéder uniquement aux pages publiques (accueil, connexion, inscription).

Utilisateur authentifié : Un visiteur connecté avec succès. Il accède à son espace personnel et gère ses notes (création, consultation, modification, archivage, suppression, recherche).

L'application ne comporte pas d'acteur **Administrateur** : la gestion des utilisateurs reste manuelle via un accès direct à la base de données. Cette simplification est volontaire et cohérente avec le périmètre d'un projet semestriel.

2. 3 Besoins fonctionnels

Les besoins fonctionnels décrivent **ce que** le système doit faire. Ils sont organisés en trois niveaux : besoins bruts, cas d'utilisation, et user stories agiles.

2. 3.1 Tableau des besoins

2. 3.2 Tableau des besoins

ID	Besoin	Description
BF-01	S'inscrire	Le visiteur saisit username, email et mot de passe; le système crée un compte.
BF-02	Se connecter	L'utilisateur saisit ses identifiants; le système ouvre une session.
BF-03	Se déconnecter	La session est détruite; retour à la page de connexion.
BF-04	Créer une note	L'utilisateur saisit titre + contenu; la note est associée à son compte.
BF-05	Consulter ses notes	Affichage paginé par date décroissante, hors archive et corbeille.
BF-06	Voir une note	Affichage du titre et du contenu complet d'une note.

Tableau 2. – Tableau des besoins fonctionnels.

ID	Besoin	Description
BF-07	Modifier une note	Édition du titre/contenu; mise à jour de updated_at.
BF-08	Archiver une note	Note masquée de la liste principale, réversible.
BF-09	Mettre à la corbeille	Note masquée, restauration possible.
BF-10	Restaurer une note	Retour de la note dans la liste principale.
BF-11	Supprimer définitivement	Suppression DELETE en base, irréversible.
ID	Besoin	Description
BF-12	Rechercher une note	Filtre par titre/contenu, résultats limités au compte courant.
BF-13	Isolation des données	Toute requête filtrée par user_id de la session.

Tableau 3. – Tableau des besoins fonctionnels.

2. 3.3 Diagramme de cas d'utilisation

La Figure Fig. 1 synthétise les cas d'utilisation et leurs liens avec les acteurs. La relation « **include** » indique que les opérations sur les notes nécessitent une authentification préalable (BF-02).

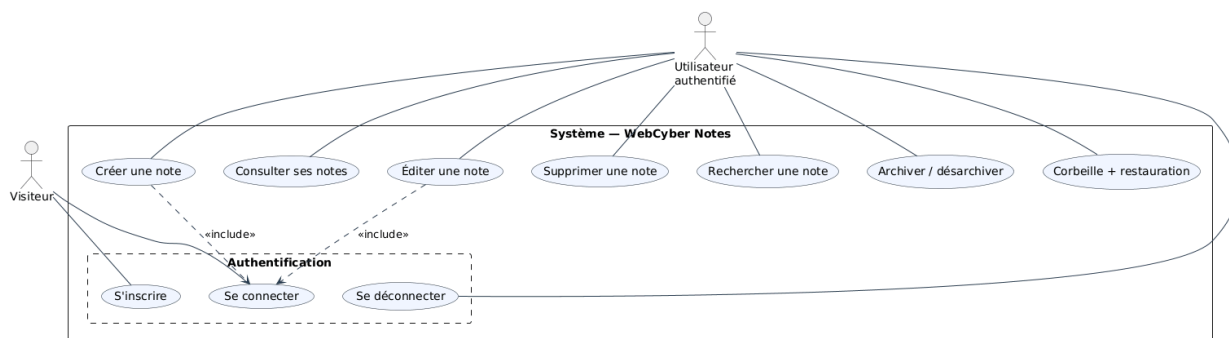


Fig. 1. – Diagramme de cas d'utilisation de WebCyber.

2. 3.4 User Stories (approche agile)

Dans une démarche agile, les besoins sont reformulés en **user stories** selon le format standard :

En tant que [acteur], **je veux** [action] **afin de** [bénéfice].

Chaque story est accompagnée de **critères d'acceptation** mesurables. Le Tableau Tableau 4 Tableau 5 présente l'ensemble des stories.

ID	Acteur	Story	Critères d'acceptation
US-01	Visiteur	Créer un compte avec username, email et mot de passe	Username 3-80 car., email valide, mot de passe ≥ 8 car.
US-02	Visiteur	Me connecter avec mes identifiants	Session créée, cookie sécurisé, redirection /notes
US-03	Utilisateur	Créer une note avec titre et contenu	Titre max 200 car., horodatage auto, user_id associé

Tableau 4. – User Stories du projet WebCyber.

ID	Acteur	Story	Critères d'acceptation
US-04	Utilisateur	Consulter la liste de mes notes	Pagination, ordre antéchronologique
US-05	Utilisateur	Voir le détail d'une note	Affichage complet, accès propriétaire uniquement
US-06	Utilisateur	Modifier une note	updated_at actualisé, validation des champs
US-07	Utilisateur	Archiver une note	Note masquée, accessible dans l'archive, réversible
US-08	Utilisateur	Supprimer (corbeille)	Note en corbeille, restaurable
US-09	Utilisateur	Restaurer une note	Retour dans liste principale, is_trashed = false
US-10	Utilisateur	Supprimer définitivement	Suppression DELETE irréversible
US-11	Utilisateur	Rechercher parmi mes notes	Filtre titre/contenu, résultats limités
US-12	Utilisateur	Isolation des données	Toute requête filtrée par user_id → 404 si accès direct

Tableau 5. – User Stories du projet WebCyber.

2. 3.5 Product Backlog

Le **Product Backlog** (Tableau 6) priorise les user stories selon leur valeur métier et leurs dépendances techniques. Les stories critiques (sécurité, isolation) ont été traitées dès les premiers sprints, conformément au principe **Security by Design**.

ID	Story	Priorité	Sprint	Statut
US-01	Inscription	Haute	3	Terminé
US-02	Connexion	Haute	3	Terminé
US-12	Isolation données	Critique	3-5	Terminé
US-03	Créer une note	Haute	3	Terminé
US-04	Lister les notes	Haute	3	Terminé
US-05	Voir une note	Haute	3	Terminé
US-06	Modifier une note	Moyenne	4	Terminé
US-07	Archiver une note	Basse	4	Terminé
US-08	Supprimer (corbeille)	Moyenne	4	Terminé
US-09	Restaurer une note	Moyenne	4	Terminé
US-10	Suppression définitive	Basse	4	Terminé
US-11	Rechercher	Basse	4	Terminé

Tableau 6. – Product Backlog du projet WebCyber.

La priorisation suit la méthode **MoSCoW** :

- **Critique** : indispensable (sécurité, isolation)
- **Haute** : nécessaire pour la démonstration
- **Moyenne** : améliore l'expérience utilisateur
- **Basse** : fonctionnalité de confort

2. 4 Besoins non fonctionnels

Les besoins non fonctionnels traduisent les **qualités attendues** du système, indépendamment des fonctionnalités.

2. 4.1 Sécurité

Exigence	Description
Confidentialité	Tout trafic client-serveur chiffré (HTTPS, TLS 1.2/1.3)
Authentification	Mots de passe hachés (PBKDF2-SHA256), jamais stockés en clair
Protection OWASP	Prévention des injections SQL, XSS, CSRF, contournement d'accès
Isolation réseau	Seuls les ports 22, 80, 443 exposés
Isolation données	Un utilisateur ne peut accéder aux notes d'un autre

Tableau 7. – Exigences de sécurité.

2. 4.2 Performance et disponibilité

Exigence	Description
Disponibilité	Redémarrage auto des conteneurs (restart: unless-stopped)
Performance	Support de 10 utilisateurs simultanés sur t2.micro (1 vCPU, 1 Gio RAM)

Tableau 8. – Exigences de performance et disponibilité.

2. 4.3 Déploiement et maintenabilité

Exigence	Description
Reproductibilité	Déploiement from scratch en < 30 minutes depuis le dépôt Git
Conteneurisation	100% Docker Compose, rien installé directement sur l'hôte
Documentation	Architecture et sécurité documentées pour reprise
Certificats TLS	Renouvellement automatique via Certbot (cron quotidien)

Tableau 9. – Exigences de déploiement et maintenabilité.

2. 4.4 Ergonomie

- Interface **responsive** (mobile/desktop) avec Tailwind CSS
- Navigation simple : barre latérale (liste, archive, corbeille)

2. 4.5 Périmètre exclu

Pour rester fidèle au cadre d'un projet semestriel, ces éléments sont **volontairement exclus** (mais mentionnés en perspectives) :

- Orchestration Kubernetes / auto-scaling
- Pipeline CI/CD complet
- WAF / services AWS avancés (GuardDuty)
- Centralisation des logs (CloudWatch, ELK)
- Monitoring (Prometheus, Grafana)

2. 5 Synthèse des exigences

Catégorie	Éléments clés
Fonctionnel	13 besoins, 12 user stories, priorisées en backlog
Sécurité	HTTPS, hachage, OWASP Top 10, isolation données
Architecture	IaaS (AWS EC2), Docker Compose, Nginx, Flask, PostgreSQL
Qualité	Déploiement reproductible, documentation, conteneurisation

Tableau 10. – Synthèse des exigences du projet.

2. 6 Conclusion du chapitre

L'analyse des besoins fait ressortir un système fonctionnellement simple mais soumis à des exigences non fonctionnelles nombreuses (sécurité, déploiement reproductible, isolation, HTTPS).

La formalisation en user stories et le **Product Backlog** structurent le développement en sprints cohérents. Le chapitre suivant traduit ces exigences en une architecture concrète : AWS, Docker, modèle de données et sécurité en profondeur.

Chapitre 3

3 CONCEPTION

3. 1 Introduction

Ce chapitre traduit les besoins du chapitre précédent en une **architecture technique** précise. Il présente successivement :

- l'architecture globale du système ;
- l'infrastructure AWS et l'architecture Docker ;
- le modèle de données ;
- les scénarios d'interaction (diagrammes de séquence) ;
- la conception de la sécurité **en profondeur** (6 couches).

3. 2 Architecture globale

L'architecture cible est résumée par la Figure Fig. 2. Le navigateur du client résout le nom de domaine `webcyber.app` via Route 53, qui retourne l'**Elastic IP** de l'instance EC2.

Pourquoi ce choix ? L'utilisation d'une Elastic IP garantit que l'adresse publique reste stable même après un redémarrage de l'instance, indispensable pour la configuration DNS et les certificats TLS.

La requête HTTPS traverse d'abord le **Security Group** (pare-feu au niveau du cloud), puis atteint le conteneur Nginx. Ce dernier assure la terminaison TLS (déchiffrement) et **proxifie** la requête vers l'application Flask. Cette dernière interroge PostgreSQL sans jamais exposer sa base directement.

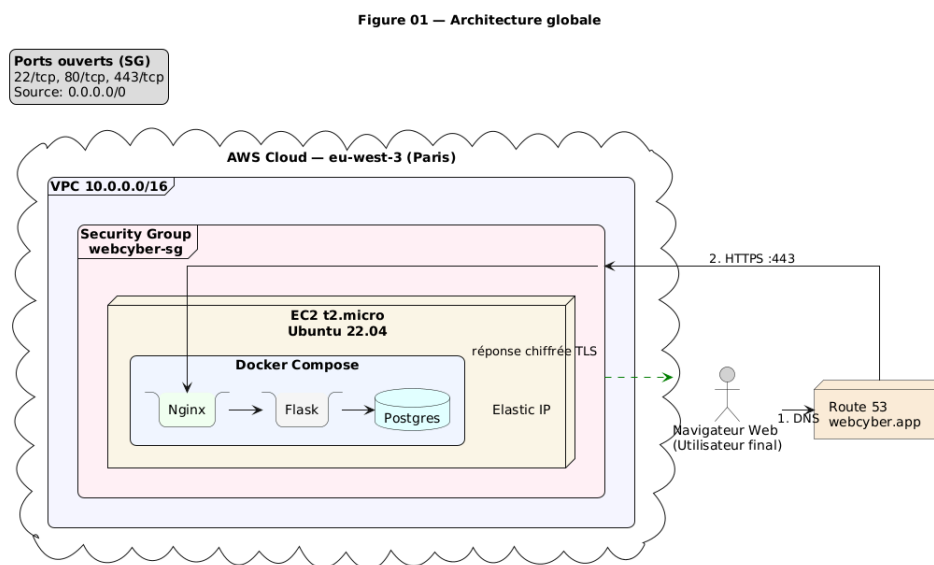


Fig. 2. – Architecture globale - vue d'ensemble.

3. 2.1 Limites de confiance (trust boundaries)

Le système définit trois limites de confiance distinctes :

1. **Frontière Internet** : entre le client et l'instance EC2. Tout trafic non HTTPS est rejeté.
2. **Frontière Docker** : entre Nginx (exposé) et les conteneurs internes (Flask, PostgreSQL). Seul Nginx peut initier des connexions vers Flask.
3. **Frontière base de données** : entre l'application et PostgreSQL. Les identifiants sont injectés via variables d'environnement.

3. 2.2 Réduction de la surface d'attaque

La surface d'attaque est minimisée par trois décisions majeures :

- **Ports exposés** : seulement 22 (SSH), 80 (HTTP), 443 (HTTPS). Les ports 5000 (Flask) et 5432 (PostgreSQL) restent fermés.
- **Pas d'administration web** : aucun panneau d'administration exposé (phpMyAdmin, Adminer, etc.).
- **Conteneurs internes** : Flask et PostgreSQL ne publient aucun port sur l'hôte. Ils ne sont accessibles que via le réseau Docker privé.

3. 2.3 Composants et responsabilités

Composant	Responsabilité
Route 53	Résolution DNS : <code>webcyber.app</code> → Elastic IP
Elastic IP	Adresse IPv4 publique fixe associée à l'EC2
EC2 (t2.micro)	Hôte Ubuntu 22.04 exécutant Docker
Security Group	Pare-feu stateful au niveau de l'EC2
Conteneur Nginx	Proxy inverse, terminaison TLS, en-têtes de sécurité
Conteneur Flask	Logique applicative, authentification, CRUD
Conteneur PostgreSQL	Persistance des données utilisateur

Tableau 11. – Composants et responsabilités.

3. 3 Infrastructure AWS

3. 3.1 Choix de la région

La région eu-west-3 (Paris) a été choisie pour :

- sa proximité géographique (latence réduite vers la Tunisie) ;
- la résidence des données (stockage en Europe, conformité RGPD) ;
- la disponibilité du **free tier** (gratuit pour 12 mois).

3. 3.2 Composants AWS retenus

La Figure Fig. 3 détaille l'organisation interne au **cloud**. Le choix du VPC par défaut simplifie le déploiement tout en offrant l'isolation réseau nécessaire pour un projet de cette envergure.

Figure 02 — Infrastructure AWS détaillée

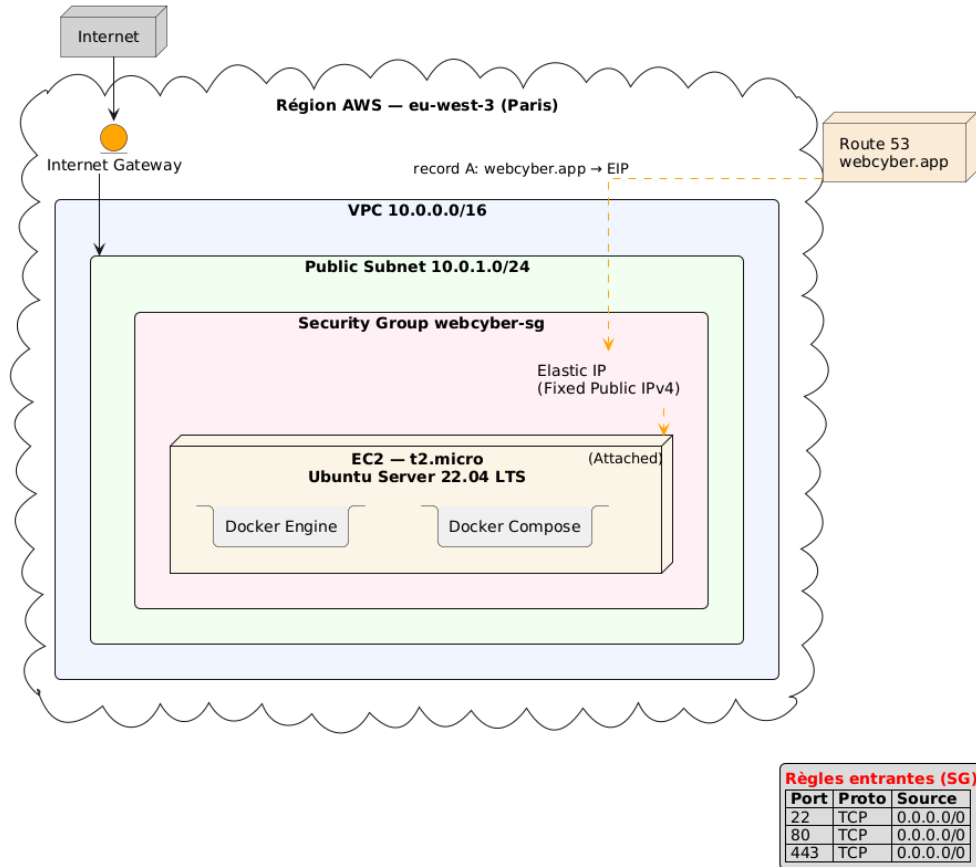


Fig. 3. – Infrastructure AWS détaillée.

3.3.3 Règles du Security Group

Le Security Group agit comme un pare-feu **stateful** : si une requête entrante est autorisée sur un port, la réponse correspondante est automatiquement autorisée en sortie.

Pourquoi ces ports uniquement ?

- **Port 22 (SSH)** : administration nécessaire, mais protégé par clé (pas de mot de passe).
- **Port 80 (HTTP)** : redirige immédiatement vers HTTPS.
- **Port 443 (HTTPS)** : trafic légitime de l'application.
- **Ports 5000 et 5432 fermés** : même en cas de fuite de configuration, la base et l'application restent inaccessibles depuis Internet.

Type	Proto- cole	Port	Source	Justification
SSH	TCP	22	0.0.0.0/0	Administration distante (clé uniquement)
HTTP	TCP	80	0.0.0.0/0	Redirection 301 vers HTTPS
HTTPS	TCP	443	0.0.0.0/0	Trafic web chiffré

Tableau 12. – Règles entrantes du Security Group webcyber-sg.

3. 4 Architecture Docker

3. 4.1 Vue d'ensemble

Trois conteneurs cohabitent sur l'instance, orchestrés par Docker Compose et reliés par un réseau **bridge** privé (webcyber-net).

Pourquoi un réseau bridge privé ? Docker crée par défaut un réseau bridge sur lequel tous les conteneurs peuvent communiquer. En créant un réseau dédié, on évite que des conteneurs tiers (futurs ou malveillants) n'accèdent à nos services.

Pourquoi seul Nginx expose des ports ? C'est le principe du **reverse proxy** : une seule entrée publique, qui protège les services internes. Même si un attaquant parvenait à exécuter du code dans Flask ou PostgreSQL, il ne pourrait pas exposer ces services directement car leurs ports ne sont pas publiés.

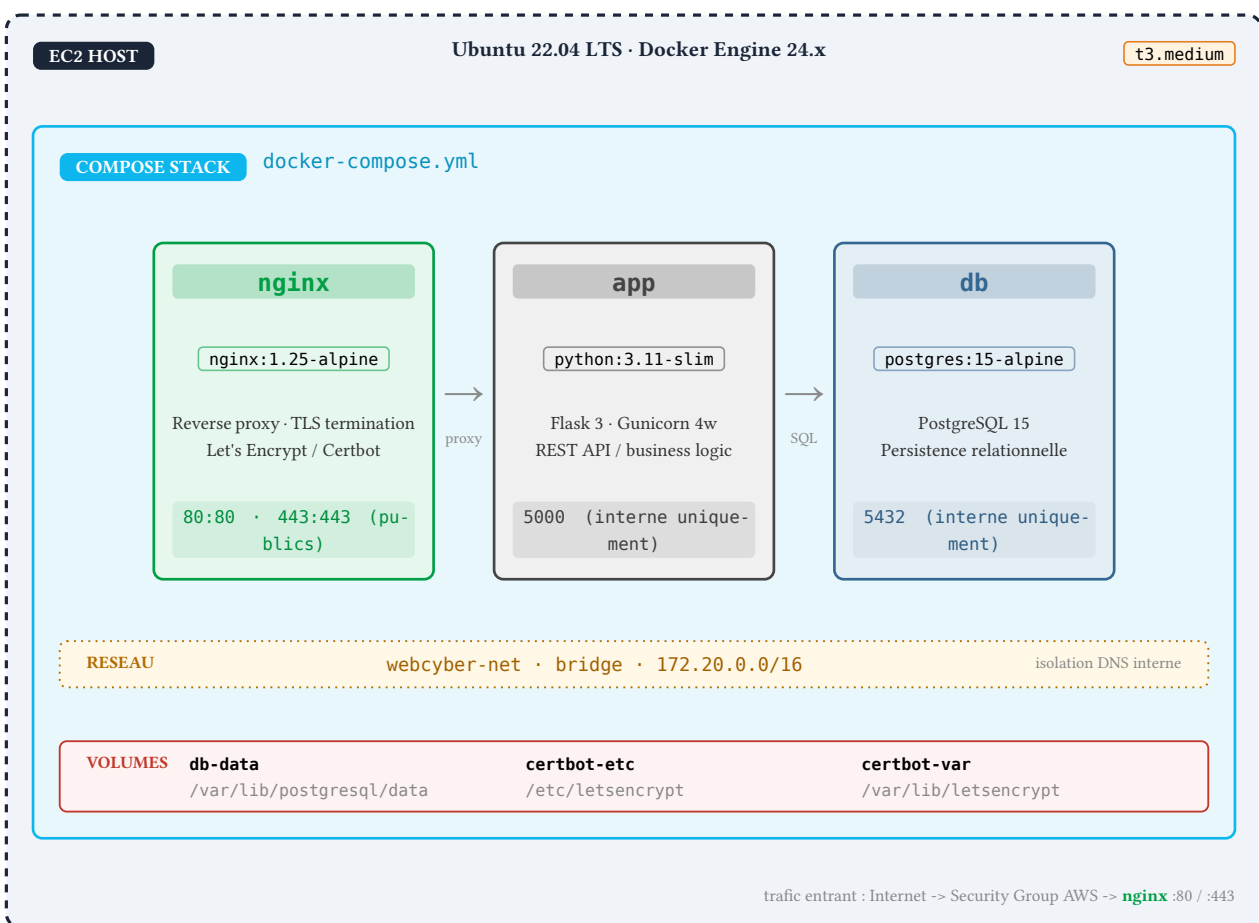


Fig. 4. – Architecture Docker - trois conteneurs sur réseau bridge.

3. 4.2 Spécifications des conteneurs

Le choix des images -alpine réduit la surface d'attaque : ces distributions Linux minimalistes contiennent moins de paquets (vulnérabilités potentielles) et pèsent 5 à 10 fois moins lourd.

Service	Image	Ports	Rôle
nginx	nginx:1.25-alpine	80, 443 (publiés)	Proxy inverse, TLS, en-têtes
app	python:3.11-slim (build)	5000 (interne)	Application Flask + Gunicorn
db	postgres:15-alpine	5432 (interne)	Base de données PostgreSQL

Tableau 13. – Spécifications des conteneurs.

3. 4.3 Justification des choix techniques

- **Nginx vs Apache** : Nginx consomme moins de mémoire et gère mieux les connexions simultanées (asynchrone). Idéal pour un t2.micro.
- **PostgreSQL vs MySQL** : PostgreSQL offre un meilleur respect des standards SQL et une réputation supérieure en matière d'intégrité des données.
- **Alpine Linux** : images minimalistes (5 Mo pour Alpine vs 200 Mo pour Debian), donc moins de vulnérabilités potentielles.

3. 5 Modèle de données

Le modèle de données est volontairement réduit à deux entités, comme le montre la Figure Fig. 5. Cette simplicité permet de concentrer l'effort de conception sur les aspects de sécurité et d'isolation.

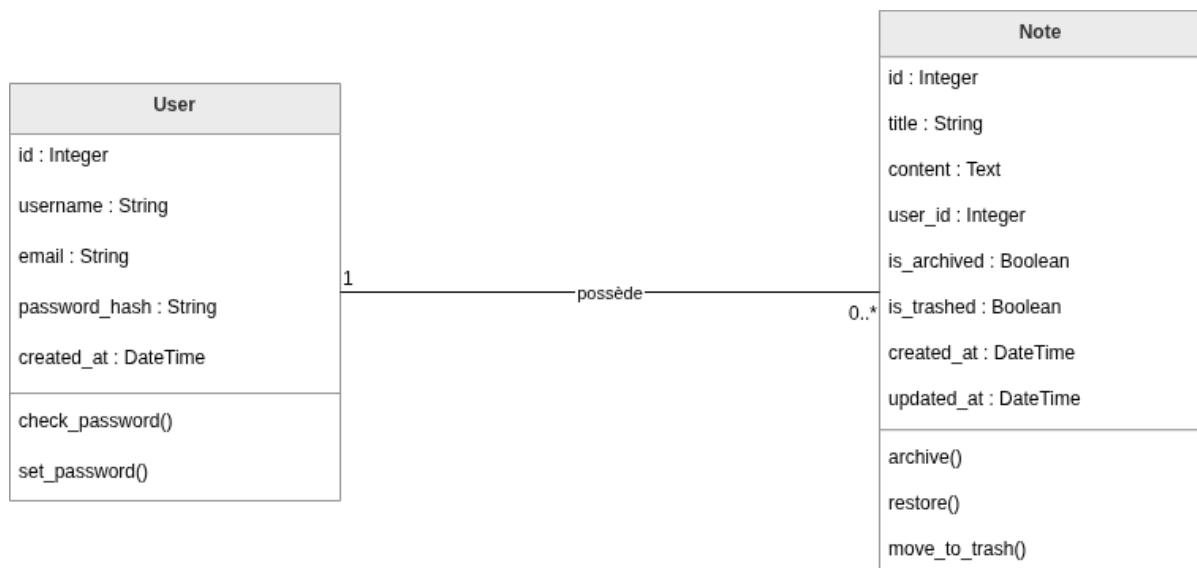


Fig. 5. – Diagramme de classes - modèle de données.

3. 5.1 Description des entités

User : représente un compte utilisateur.

- `id` : clé primaire, auto-générée
- `username` : unique, 3-80 caractères
- `email` : unique, format validé
- `password_hash` : hachage PBKDF2-SHA256 (jamais en clair)
- `created_at` : horodatage automatique

Un utilisateur peut posséder zéro à n notes (relation **one-to-many**).

Note : représente le contenu textuel.

- `id` : clé primaire
- `title` : titre (max 200 caractères)
- `content` : corps textuel
- `user_id` : clé étrangère vers `User.id`
- `is_archived` : archive douce (true/false)
- `is_trashed` : corbeille réversible (true/false)
- `created_at` / `updated_at` : horodatages automatiques

Une note appartient à exactement un utilisateur.

3. 5.2 Considérations de sécurité

- **Hachage** : `password_hash` stocke uniquement l’empreinte du mot de passe (PBKDF2-SHA256), jamais le texte clair.
- **Unicité** : les contraintes UNIQUE sur `username` et `email` empêchent les doublons et limitent l’énumération.
- **Filtrage** : toute requête SQL sur notes inclut `WHERE user_id = current_user.id` (isolation stricte).
- **Intégrité** : la clé étrangère avec `ON DELETE CASCADE` évite les notes orphelines.

3. 6 Diagrammes de séquence

3. 6.1 Authentification

Le scénario d'authentification (Figure Fig. 6) met en jeu cinq acteurs : utilisateur, navigateur, Nginx, Flask et PostgreSQL.

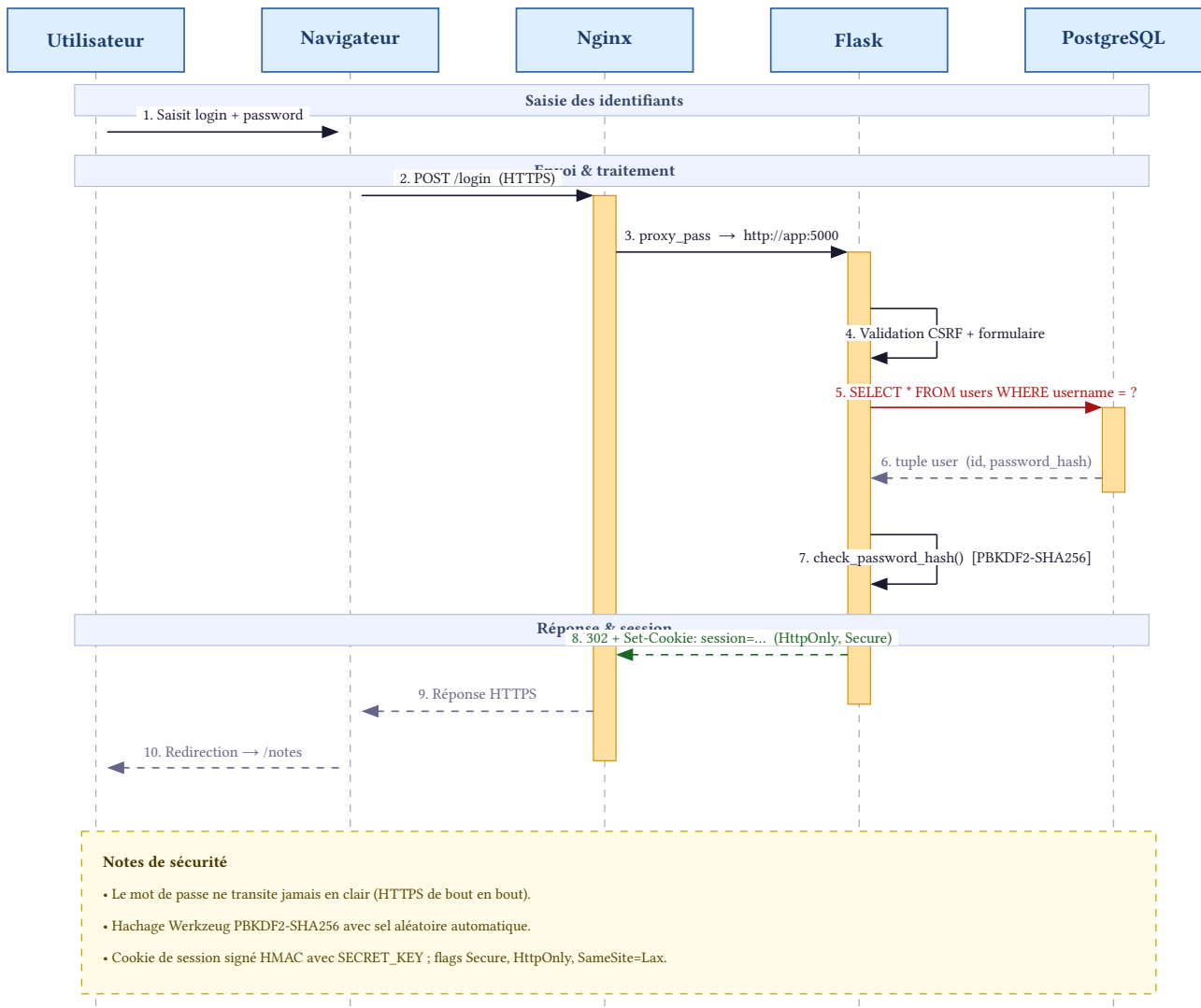


Fig. 6. – Diagramme de séquence - authentification.

Points de sécurité :

1. Le mot de passe ne quitte jamais le tunnel TLS
2. Comparaison **constant-time** via `check_password_hash`
3. Cookie de session signé (HMAC) avec attributs Secure, HttpOnly, SameSite=Lax

3. 6.2 Création d'une note

Le scénario de création d'une note (Figure Fig. 7) montre le contrôle de session (@login_required), la vérification CSRF, et l'insertion en base avec association à l'utilisateur courant.

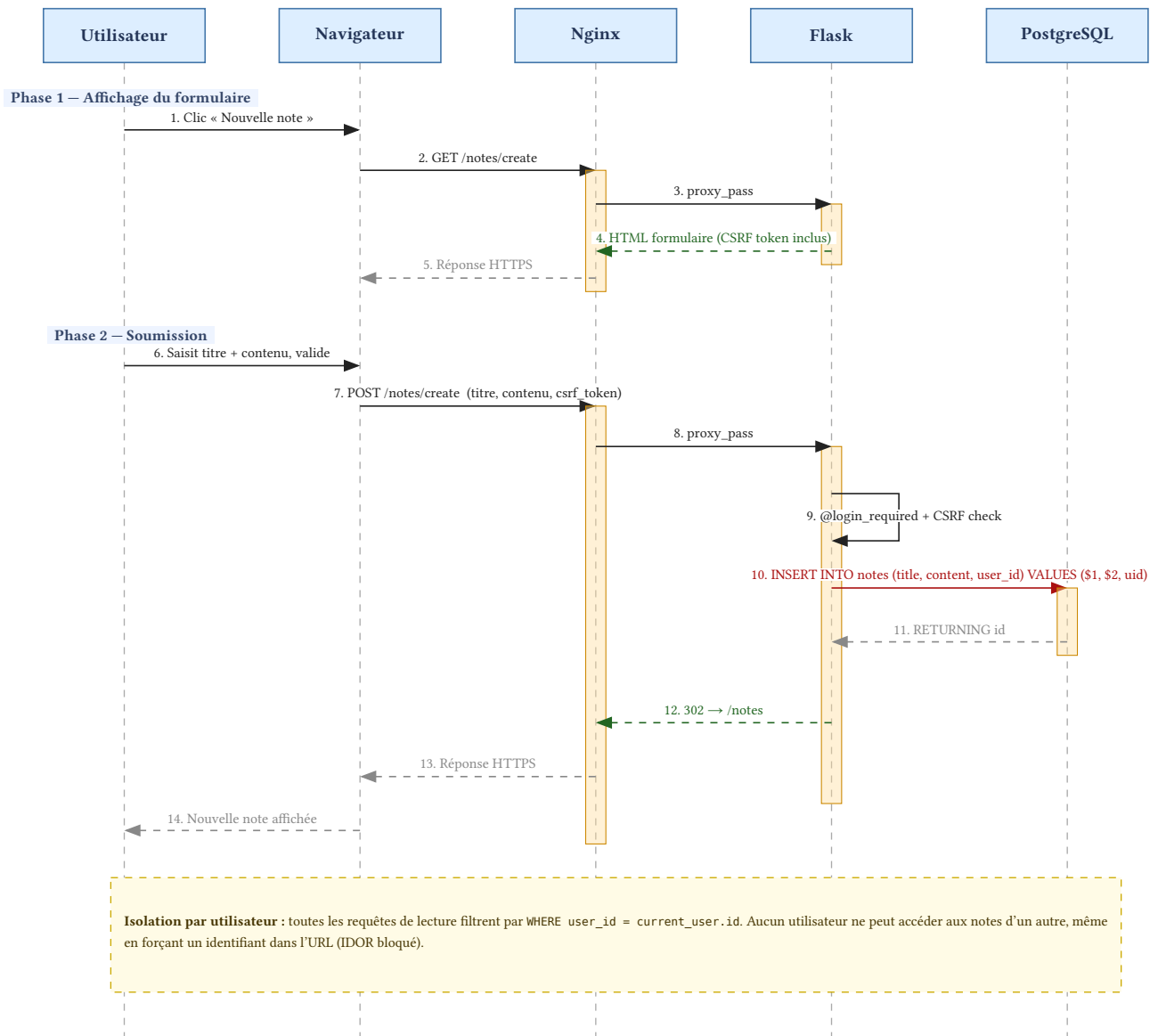


Fig. 7. – Diagramme de séquence - création d'une note.

3. 7 Conception de la sécurité - défense en profondeur

La sécurité du système ne repose pas sur une mesure unique mais sur **six couches empilées**, chacune apportant une garantie spécifique. Si un attaquant compromet une couche, les cinq autres restent opérationnelles pour bloquer l'attaque.

La Figure Fig. 8 illustre cette approche.

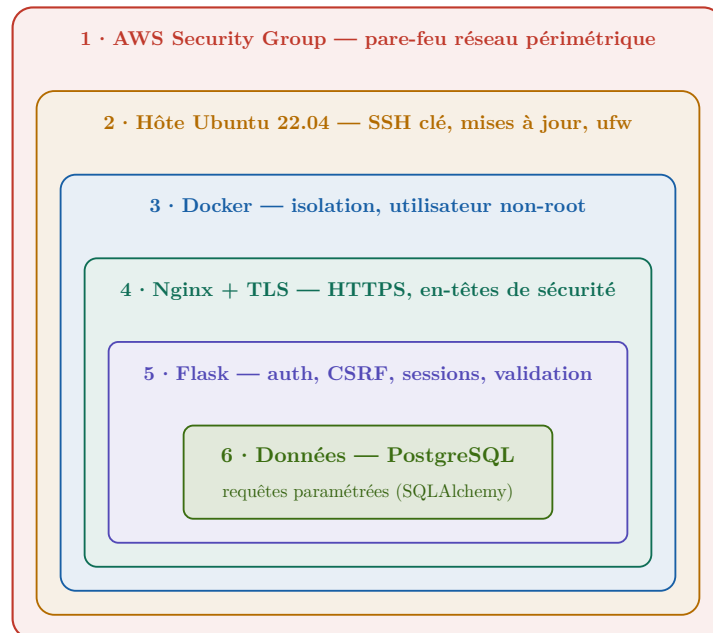


Fig. 8. – Six couches de sécurité (defense-in-depth).

3. 7.1 Couche 1 - Security Group (AWS)

Rôle : Pare-feu périmétrique filtrant le trafic entrant au niveau de l'instance EC2.

Pourquoi cette couche est essentielle ? Elle bloque les attaques au niveau du réseau, avant même qu'elles n'atteignent l'OS ou Docker. Même si l'application a une faille, le port 5000 (Flask) reste inaccessible.

Règle	Valeur
Ports ouverts	22 (SSH), 80 (HTTP), 443 (HTTPS)
Ports fermés	5000 (Flask), 5432 (PostgreSQL)
Nature	Stateful (réponses auto-autorisées)

Tableau 14. – Configuration du Security Group.

3. 7.2 Couche 2 - Système hôte (EC2)

Rôle : Configuration sécurisée de l'instance Ubuntu hébergeant Docker.

Pourquoi ces configurations ? `PermitRootLogin no` empêche les attaques par force brute sur le compte root. L'authentification par clé (pas de mot de passe) rend les attaques par dictionnaire impossibles. Les mises à jour automatiques corrigent les vulnérabilités connues sans intervention manuelle.

Configuration	Valeur
SSH root login	PermitRootLogin no
Authentification	Clé SSH uniquement (pas de mot de passe)
Mises à jour	unattended-upgrades automatiques
Utilisateur applicatif	ubuntu (non-root)

Tableau 15. – Configuration du système hôte.

3. 7.3 Couche 3 - Isolation Docker

Rôle : Isolation réseau et processus entre les trois conteneurs.

Principe clé : « Un conteneur = un service ». Si Flask est compromis, l’attaquant ne peut pas accéder à PostgreSQL car les ports ne sont pas publiés. Si PostgreSQL est compromis, l’attaquant ne peut pas sortir du réseau Docker privé.

Mesure	Description
Réseau	Bridge privé webcyber-net (isolation réseau)
Exposition ports	app et db sans port publié vers l’hôte
Utilisateur	appuser (UID 1000) non-privilegié dans le conteneur
Volumes	Données persistantes isolées du système hôte

Tableau 16. – Mesures d’isolation Docker.

3. 7.4 Couche 4 - Nginx + TLS

Rôle : Proxy inverse assurant la terminaison TLS et l’injection d’en-têtes de sécurité.

Pourquoi TLS 1.2 et 1.3 uniquement ? Les versions antérieures (TLS 1.0, 1.1) ont des vulnérabilités connues (POODLE, BEAST). Le profil **Mozilla Intermediate** garantit la compatibilité avec 99% des navigateurs tout en excluant les algorithmes faibles.

Configuration	Valeur
TLS versions	1.2 et 1.3 uniquement (versions sécurisées)
Certificat	Let’s Encrypt (renouvellement automatique)
Redirection	HTTP → HTTPS (301 permanent)
En-têtes sécurité	HSTS, CSP, X-Frame-Options, X-Content-Type-Options
Version masking	server_tokens off (masque la version Nginx)

Tableau 17. – Configuration Nginx et TLS.

3. 7.5 Couche 5 - Application Flask

Rôle : Logique métier, authentification et contrôle d’accès aux données.

Pourquoi PBKDF2-SHA256 ? C’est l’algorithme recommandé par l’OWASP pour le stockage des mots de passe. Le sel automatique rend les attaques par tables arc-en-ciel inefficaces.

Protection	Mécanisme
Mots de passe	PBKDF2-SHA256 avec sel automatique
Session	Signée par SECRET_KEY (32 octets aléatoire)
CSRF	Flask-WTF sur tous les formulaires
Validation	WTForms (longueur, type, présence)
Routes privées	Décorateur @login_required
Isolation données	Filtre user_id sur 100% des requêtes

Tableau 18. – Mesures de sécurité de l'application Flask.

3. 7.6 Couche 6 - Base de données

Rôle : Persistance des données avec isolation réseau et requêtes paramétrées.

Pourquoi des requêtes paramétrées ? Elles sont la seule défense fiable contre les attaques par injection SQL. La concaténation de chaînes est totalement prohibée.

Mesure	Description
Utilisateur	webcyber_user (non-superutilisateur)
Mot de passe	Long (>20 car.), aléatoire, injecté via .env
Exposition	Aucun port publié vers l'hôte (Docker interne)
Requêtes	Paramétrées (SQLAlchemy) - pas de concaténation SQL

Tableau 19. – Mesures de sécurité de la base de données.

3. 8 Récapitulatif de la conception

Couche	Technologie	Éléments clés
DNS	Route 53	Zone hébergée, résolution
Cloud	AWS EC2	t2.micro, Security Group
Orchestration	Docker Compose	3 conteneurs, réseau bridge
Proxy	Nginx	TLS, en-têtes, redirection
Application	Flask	Authentification, CRUD, sessions
Base de données	PostgreSQL	Persistance, isolation
Sécurité	6 couches	Defense-in-depth

Tableau 20. – Synthèse de l'architecture technique.

3. 9 Conclusion du chapitre

La conception ainsi décrite est volontairement **minimale mais complète** :

- un seul serveur (EC2 t2.micro)
- trois conteneurs (Nginx, Flask, PostgreSQL)
- deux entités (User, Note)
- six couches de sécurité empilées

L'approche **defense-in-depth** garantit que la compromission d'une couche n'entraîne pas celle du système entier. Le choix d'exposer uniquement Nginx, la création d'un réseau Docker privé, et le filtrage systématique par `user_id` constituent les piliers de cette sécurité.

Le chapitre suivant présente la réalisation concrète de cette conception et les étapes de déploiement sur AWS.

Chapitre 4

4 RÉALISATION ET DÉPLOIEMENT

4. 1 Introduction

Ce chapitre décrit comment la conception du chapitre 3 a été **transformée en infrastructure opérationnelle**. L'accent est mis sur l'automatisation, la reproductibilité et la sécurité opérationnelle, plutôt que sur une liste d'étapes manuelles.

Quatre principes guident cette réalisation :

- **Infrastructure as Code (IaC)** : L'infrastructure AWS est décrite dans des fichiers Terraform versionnés.
- **Conteneurisation** : Chaque service tourne dans son propre conteneur Docker, isolé et reproductible.
- **CI/CD automatisé** : Un pipeline GitHub Actions déploie automatiquement à chaque push.
- **Sécurité par défaut** : HTTPS, pare-feu, isolation réseau, et secrets injectés (jamais versionnés).

4. 2 Vue d'ensemble de l'infrastructure

La Figure Fig. 9 présente l'architecture de déploiement complète. Trois environnements distincts coexistent :

1. **Poste de développement** : Code Python, Dockerfile, docker-compose.yml, Terraform. Tout est versionné dans Git.
2. **CI/CD (GitHub Actions)** : Pipeline automatisé qui construit l'image Docker, la pousse vers GHCR, et se connecte à l'instance EC2 pour le déploiement.
3. **Production (AWS)** : Instance EC2 (Ubuntu 24.04) exécutant trois conteneurs (Nginx, Flask, PostgreSQL) orchestrés par Docker Compose.

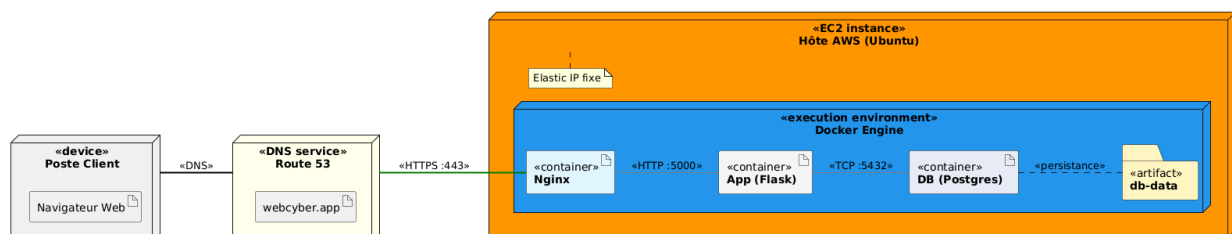


Fig. 9. – Diagramme de déploiement UML - artefacts physiques et logiques.

Le flux de déploiement suit une chaîne d'automatisation complète : `git push` → GitHub Actions → build image → push GHCR → SSH EC2 → docker compose pull → up. Aucune intervention manuelle sur le serveur n'est nécessaire après l'initialisation.

4. 3 Stack technologique

Le choix des technologies privilégie la **robustesse** et la **transparence pédagogique** plutôt que la surenchère technique.

Couche	Choix	Justification
Cloud / IaaS	AWS EC2	Standard du marché, free tier, documentation abondante
IaC	Terraform	Déclaratif, reproductible, versionnable, auditable
Conteneurs	Docker + Compose	Reproductibilité, isolation, portabilité
CI/CD	GitHub Actions	Intégré au dépôt, gratuit pour open-source
Sécurité IaC	tfsec	Analyse statique Terraform avant déploiement
Proxy / TLS	Nginx	Léger, robuste, terminaison TLS éprouvée
Application	Flask + Unicorn	Léger, transparent, pédagogique
Base de données	PostgreSQL	Robuste, respect des standards SQL
Front-end	Tailwind CSS	Responsive, productif
Registre images	GHCR (GitHub Container Registry)	Intégré à GitHub, pas de compte Docker Hub

Tableau 21. – Synthèse de la pile technologique.

4. 4 Infrastructure as Code avec Terraform

4. 4.1 Philosophie

L'infrastructure AWS n'est pas créée manuellement via la console. Elle est décrite **en code** dans des fichiers Terraform versionnés. Cette approche apporte :

- **Reproductibilité** : `terraform apply` produit la même infrastructure partout (dev, prod, disaster recovery).
- **Auditabilité** : Toute modification passe par une Pull Request et laisse une trace dans Git.
- **Validation automatique** : `terraform plan` montre les changements avant application.
- **Analyse statique** : `tfsec` détecte les mauvaises configurations avant déploiement (chapitre 5).

4. 4.2 Ressources provisionnées

Le module Terraform définit trois ressources clés :

Ressource	Configuration et justification
AMI	Ubuntu 24.04 LTS (dernière version Canonical, supportée jusqu'en 2029)
Instance EC2	t2.micro (free tier), IMDSv2 obligatoire (<code>http_tokens = "required"</code>)
Security Group	Ports 22 (SSH), 80 (HTTP), 443 (HTTPS) ouverts ; ports 5000 et 5432 fermés
Elastic IP	Adresse fixe associée (nécessaire pour DNS et certificats TLS)
Provisionnement	Installation Docker via <code>remote-exec</code> (script bash idempotent)

Tableau 22. – Ressources Terraform provisionnées.

Pourquoi IMDSv2 obligatoire ? IMDS (Instance Metadata Service) permet de récupérer des informations sur l'instance. IMDSv1 est vulnérable aux attaques SSRF. IMDSv2 ajoute une protection par jeton (PUT), ce qui rend l'attaque beaucoup plus difficile.

Pourquoi t2.micro et pas t3.micro ? Le free tier AWS inclut 750h/mois de t2.micro. t3.micro n'est pas gratuit. Pour un projet académique, le surcoût n'est pas justifié.

4. 5 Conteneurisation et orchestration

4. 5.1 Philosophie

Les trois services sont conteneurisés pour garantir :

- **Reproductibilité** : L'application s'exécute à l'identique sur le poste de développement, sur EC2, ou sur tout autre hôte.
- **Isolation** : Une compromission de l'application n'impacte pas l'hôte (et réciproquement).
- **Portabilité** : Le même `docker-compose.yml` fonctionne partout (local, cloud, autre fournisseur).

4. 5.2 Structure des conteneurs

Service	Image	Ports	Rôle et sécurité
nginx	nginx:1.25-alpine	80, 443 (publiés)	Proxy inverse, terminaison TLS, en-têtes sécurité
app	Build local → GHCR	5000 (interne)	Flask + Gunicorn, utilisateur non-root (UID 1000)
db	postgres:15-alpine	5432 (interne)	Base de données, volume persistant, mot de passe via <code>.env</code>

Tableau 23. – Spécifications des conteneurs.

4. 5.3 Points de sécurité sur les conteneurs

- **Image Alpine** : Réduit la surface d'attaque (5 Mo au lieu de 200 Mo pour une Debian classique).
- **Utilisateur non-root** : Le conteneur app exécute les processus sous `appuser` (UID 1000), pas sous `root`.
- **Réseau bridge privé** : `webcyber-net` isole les conteneurs. Seul Nginx a des ports publiés sur l'hôte.
- **Secrets via .env** : Mots de passe injectés au runtime, jamais écrits en dur dans le code ou l'image.

4. 5.4 Réseau : pourquoi seul Nginx est exposé ?

C'est le principe du **reverse proxy** et de l'**exposition minimale** :

- Le navigateur ne parle qu'à Nginx (port 80/443).
- Nginx communique avec Flask via le réseau interne Docker.
- Flask communique avec PostgreSQL via le même réseau interne.

Si un attaquant compromet Flask ou PostgreSQL, il ne peut pas exposer ces services directement car leurs ports ne sont pas publiés. Pour sortir du réseau Docker, il devrait compromettre Nginx (qui est l'unique passerelle).

4. 6 Déploiement continu (CI/CD)

4. 6.1 Philosophie

Le déploiement manuel (copier des fichiers via SCP, exécuter des commandes SSH) est source d'erreurs et non reproductible. L'automatisation complète via GitHub Actions garantit :

- **Rapidité** : Un `git push` suffit à livrer l'application.
- **Traçabilité** : Chaque déploiement est associé à un commit.
- **Gate de sécurité** : tfsec bloque les mauvaises configurations.
- **Rollback facile** : Repartir d'un commit antérieur.

4. 6.2 Architecture du pipeline

Le pipeline GitHub Actions est déclenché sur chaque push sur la branche main. Il comporte trois jobs séquentiels :

No	Job	Action et objectif
1	infra-scan	tfsec sur terraform/. Bloque si HIGH/CRITICAL (gate sécurité IaC)
2	build-and-push	Docker build + push vers GHCR avec tags latest et sha-\$GITHUB_SHA
3	deploy	SSH vers EC2 → docker compose pull → up -d → nettoyage images anciennes

Tableau 24. – Jobs du pipeline GitHub Actions.

4. 6.3 Gestion des secrets

Aucun secret n'est stocké dans le dépôt Git. Tous les secrets sont injectés via GitHub Secrets :

Secret	Usage
EC2_HOST	Adresse IP publique de l'instance (Elastic IP)
EC2_USER	ubuntu (utilisateur SSH standard sur AMI Ubuntu)
EC2_SSH_KEY	Clé privée vockey.pem (authentification SSH)
ENV_FILE	Fichier .env complet (SECRET_KEY, mots de passe PostgreSQL)

Tableau 25. – Secrets configurés dans GitHub Actions.

Pourquoi injecter le fichier .env entier ? Le secret ENV_FILE contient le contenu exact du fichier .env. Sur l'EC2, le job deploy le recrée et le passe à docker compose. Ainsi, aucune donnée sensible ne circule en clair et rien n'est versionné.

4. 7 Configuration DNS et HTTPS

4. 7.1 DNS avec Route 53

Deux enregistrements A sont créés dans la zone hébergée `webcyber.app` :

Nom	Type	Valeur	TTL
<code>webcyber.app</code>	A	Elastic IP (EC2)	300
<code>www.webcyber.app</code>	A	Elastic IP (EC2)	300

Tableau 26. – Enregistrements DNS de la zone hébergée Route 53.

Pourquoi l'Elastic IP est-elle nécessaire ? Une instance EC2 redémarrée peut changer d'adresse IP publique. L'Elastic IP est fixe, ce qui évite de devoir modifier les enregistrements DNS manuellement à chaque redémarrage (sans quoi les certificats TLS deviendraient invalides).

4. 7.2 TLS avec Let's Encrypt

Pourquoi Let's Encrypt plutôt qu'un certificat auto-signé ?

- Un certificat auto-signé déclenche un avertissement rouge dans tous les navigateurs (le projet paraîtrait non sécurisé).
- Let's Encrypt est gratuit, reconnu par 100% des navigateurs, et s'intègre automatiquement via Certbot.

Le certificat est obtenu au premier déploiement via Certbot en mode webroot. Une tâche cron quotidienne vérifie et renouvelle automatiquement le certificat (valide 90 jours). Le proxy Nginx est rechargé après renouvellement.

4. 7.3 Configuration Nginx

Nginx joue trois rôles critiques :

1. **Terminaison TLS** : Nginx gère les certificats, déchiffre le trafic, et forward la requête en clair à Flask (uniquement sur le réseau Docker interne).
2. **Redirection HTTP → HTTPS** : Tout trafic sur port 80 reçoit une redirection 301 (permanente) vers la version HTTPS.
3. **Injection d'en-têtes de sécurité** : HSTS, CSP, X-Frame-Options, etc.

Les en-têtes de sécurité suivants sont ajoutés à toutes les réponses :

En-tête	Protection apportée
<code>Strict-Transport-Security</code>	Force HTTPS pendant 1 an (HSTS) ; empêche le SSL stripping
<code>Content-Security-Policy</code>	Limite les sources autorisées (scripts, styles, polices) ; défense XSS
<code>X-Frame-Options</code>	Empêche le clickjacking (bloque l'iframe)
<code>X-Content-Type-Options</code>	Empêche le MIME sniffing (nosniff)
<code>Referrer-Policy</code>	Limite la fuite d'URL via l'en-tête Referer
<code>server_tokens off</code>	Masque la version de Nginx (moins d'informations pour l'attaquant)

Tableau 27. – En-têtes de sécurité HTTP injectés par Nginx.

4. 8 Flux d'une requête HTTPS

La Figure Fig. 10 synthétise le trajet d'une requête depuis le navigateur jusqu'à la base de données, avec la distinction fondamentale entre :

- **Partie chiffrée** : Navigateur → Nginx (TLS 1.3). Même un attaquant qui écoute le réseau ne peut pas lire la requête.
- **Partie en clair (mais isolée)** : Nginx → Flask → PostgreSQL (sur le réseau Docker interne, non accessible depuis Internet).

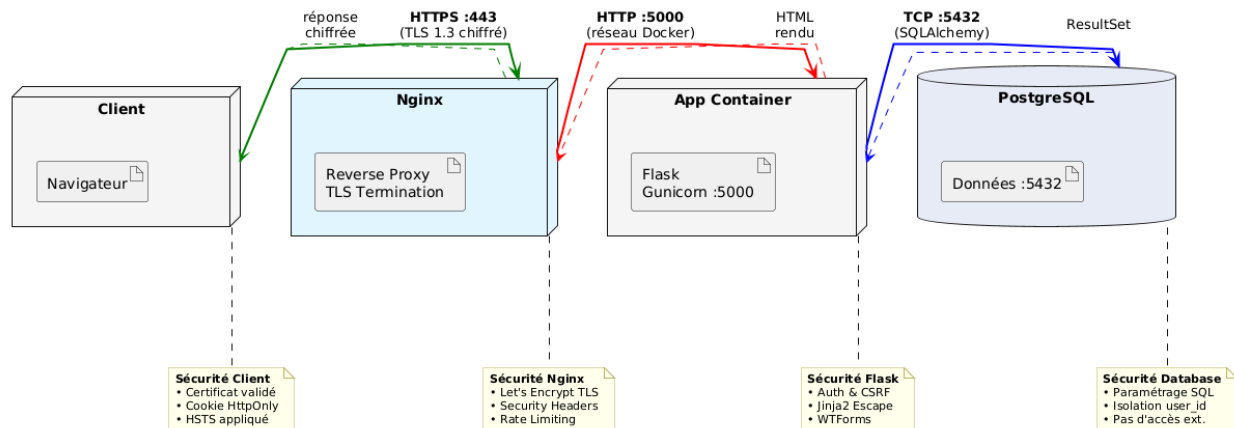


Fig. 10. – Flux d'une requête HTTPS, des en-têtes au stockage.

Ce découpage suit le principe **security by design** : le chiffrement est appliqué exactement là où il est nécessaire (trafic exposé à Internet). À l'intérieur du réseau Docker, le trafic est considéré comme confiance (mais reste isolé de l'extérieur).

4. 9 Vérification post-déploiement

Après un déploiement réussi, trois vérifications sont effectuées automatiquement (et manuellement dans ce projet) :

1. **DNS** : `dig webcyber.app` confirme la résolution vers l'Elastic IP.
2. **Certificat** : `curl -v https://webcyber.app` montre un certificat Let's Encrypt valide (pas d'avertissement).
3. **Conteneurs** : `docker compose ps` sur l'EC2 confirme que les trois services sont Up.

Les résultats détaillés des tests fonctionnels et de sécurité sont présentés au Chapitre 5.

4. 10 Interfaces de l'application

Les figures suivantes illustrent l'interface utilisateur de l'application déployée.

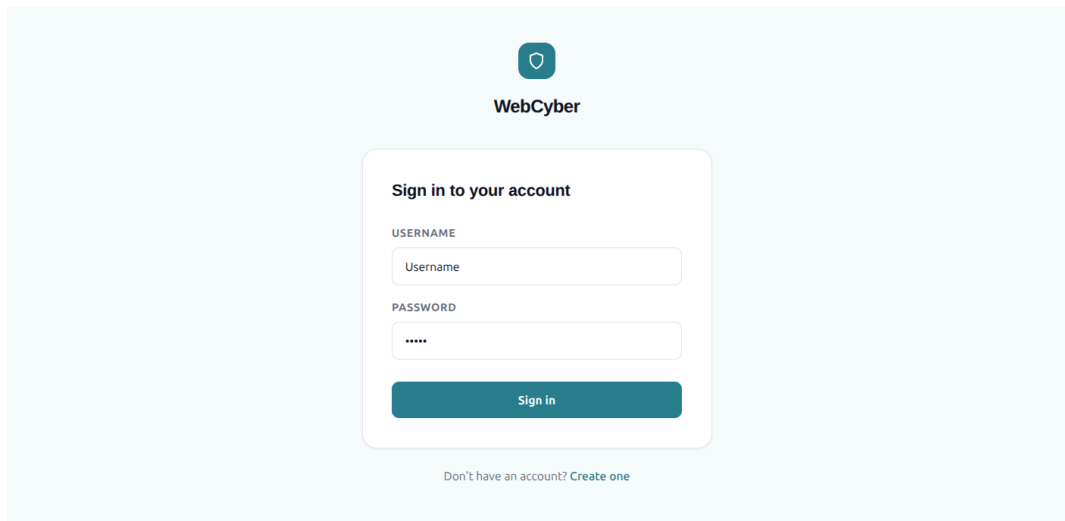


Fig. 11. – Interface de connexion / inscription (Tailwind CSS, responsive).

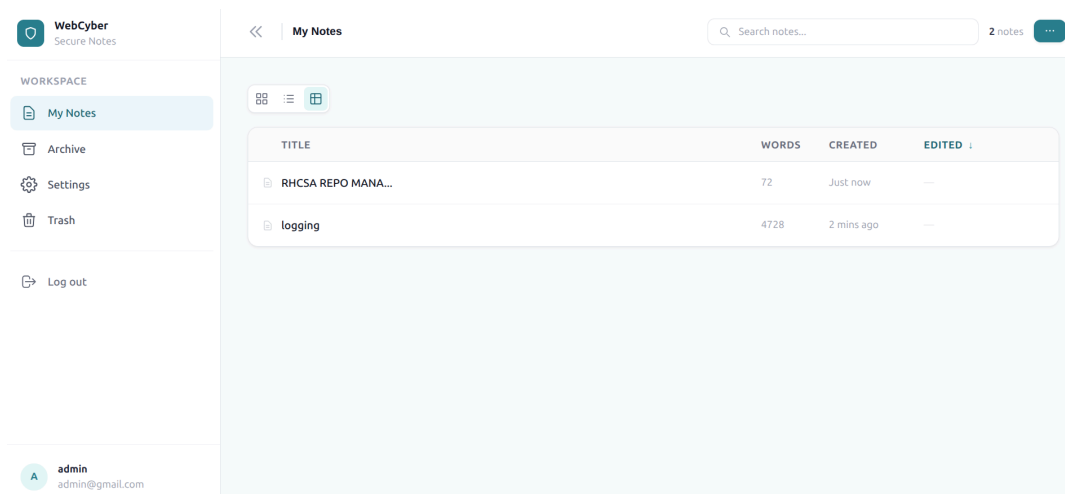


Fig. 12. – Interface de gestion des notes (liste, création, édition, archive, corbeille).

L'interface est responsive (mobile/desktop) et utilise Tailwind CSS pour un rendu moderne sans surcharge de code CSS personnalisé.

4. 11 Conclusion du chapitre

La réalisation s'appuie sur quatre piliers d'automatisation :

- **Infrastructure as Code** (Terraform) : L'infrastructure AWS est versionnée, auditable et reproductible.
- **Conteneurisation** (Docker) : Chaque service est isolé, portabilité garantie.
- **CI/CD** (GitHub Actions) : Déploiement automatisé sur git push, avec gate de sécurité tfsec.
- **Configuration automatisée** (Certbot, cron) : Renouvellement TLS sans intervention humaine.

Le résultat est un déploiement **reproductible, sécurisé par construction**, et accessible en HTTPS sur `https://webcyber.app`. Le chapitre suivant valide fonctionnellement et sécuritairement cette infrastructure.

Chapitre 5

5 TESTS ET ANALYSE DE SÉCURITÉ

5. 1 Introduction

Ce chapitre présente la dernière phase du projet : la **validation** fonctionnelle de l'application et l'**audit de sécurité** du déploiement. La méthodologie suit six phases complémentaires couvrant aussi bien le code applicatif que l'infrastructure.

L'objectif n'est pas seulement de détecter des vulnérabilités, mais de démontrer que chaque risque identifié dans la conception (chapitre 3) a été correctement traité ou documenté.

5. 2 Stratégie de test

Phase	Type	Outils	Objectif
1	Fonctionnel	Navigateur, curl	Vérifier les cas d'utilisation
2	Analyse statique IaC	tfsec	Détecter les défauts de configuration Terraform avant déploiement
3	Infrastructure	docker, dig, curl	Confirmer le déploiement
4	Scan de ports	Nmap	Limiter la surface d'attaque
5	Scan applicatif web	Nikto, curl	Détecter les mauvaises configurations HTTP
6	Évaluation TLS	SSL Labs, testssl.sh	Mesurer la qualité du chiffrement

Tableau 28. – Phases de la campagne de validation.

5. 3 Tests fonctionnels

5. 3.1 Inscription et authentification

Risque évalué : Un attaquant pourrait créer des comptes en masse (inscription) ou contourner l'authentification (connexion).

Résultat attendu : Seules les inscriptions valides aboutissent ; les tentatives avec des données invalides ou dupliquées sont rejetées.

No	Cas	Entrée	Résultat attendu
1	Inscription valide	user/email/mdp corrects	Compte créé, redirection /login
2	Username existant	username déjà pris	Erreur, pas de doublon
3	Mot de passe court	< 8 caractères	Erreur de validation
4	Champs vides	blanc	Erreur de validation

Tableau 29. – Tests d'inscription.

No	Cas	Entrée	Résultat attendu
1	Identifiants valides	user/mdp corrects	Redirection /notes
2	Mauvais mot de passe	mdp erroné	Message d'erreur
3	Utilisateur inconnu	username inexistant	Message d'erreur
4	Accès direct sans auth	GET /notes	Redirection /login
5	Déconnexion	clic « se déconnecter »	Session détruite, retour /login

Tableau 30. – Tests d'authentification.

Résultat : Tous les tests passent. Les contraintes de validation sont appliquées côté serveur (non contournables).

5. 3.2 Gestion des notes (CRUD)

Risque évalué : Un utilisateur pourrait manipuler des notes appartenant à d'autres utilisateurs (IDOR - Insecure Direct Object Reference).

Mitigation : Chaque requête SQL inclut systématiquement `WHERE user_id = current_user.id`.

No	Cas	Résultat attendu
1	Création de note	Note dans la liste
2	Détail d'une note	Titre + contenu complet affichés
3	Édition	Contenu mis à jour, updated_at actualisé
4	Archivage	Disparaît de /notes, apparaît dans /archive
5	Mise à la corbeille	Disparaît, apparaît dans /trash
6	Restauration	Revient dans la liste principale
7	Suppression définitive	Disparaît de partout, irrécupérable
8	Recherche par titre	Résultats filtrés
9	Création sans titre	Erreur de validation

Tableau 31. – Tests CRUD sur les notes.

5. 3.3 Test critique : isolation des données

Risque évalué : Violation de la confidentialité des données (un utilisateur lit les notes d'un autre).

Résultat attendu : 404 Not Found pour toute tentative d'accès à une note non autorisée.

No	Scénario	Résultat attendu
1	Utilisateur A crée la note id=42	A voit la note dans sa liste
2	Utilisateur B se connecte	B ne voit pas la note de A
3	B accède à /notes/42 directement	404 Not Found
4	B tente POST /notes/42/edit	404 Not Found, aucune modification

Tableau 32. – Tests d'isolation par utilisateur.

Résultat : L'application bloque toute tentative grâce au filtre `filter_by(user_id=current_user.id)` systématique combiné à `first_or_404()`.

5. 4 Analyse statique de l'IaC avec tfsec

5. 4.1 Principe et positionnement

tfsec est un outil d'analyse statique (SAST) spécialisé dans le code Terraform. Il examine la configuration **avant** tout déploiement et signale les défauts courants : ports trop largement ouverts, absence de chiffrement, métadonnées IMDSv2 non protégées, etc.

Pourquoi cette phase est critique ? Une mauvaise configuration d'infrastructure est plus dangereuse qu'une faille applicative : elle expose directement les ressources cloud à Internet.

5. 4.2 Résultats et décisions

L'analyse initiale a remonté trois constats. Chaque **finding** a été analysé et a reçu une décision explicite :

Règle	Sévé- rité	Description	Décision et justification
aws-ec2-no-public-ingress-sgr	HIGH	IMDSv2 non requis sur l'instance	Corrigée : ajout du bloc <code>metadata_options { http_tokens = "required" }</code>
aws-ec2-enable-at-rest-encryption	HIGH	Volume root non chiffré	Acceptée et documentée : projet académique sur AWS Academy, pas de données sensibles au repos. Exclusion par <code>tfsec:ignore</code>
aws-ec2-add-description-to-security-group	LOW	Le security group utilise la description par défaut	Acceptée et documentée : finding cosmétique de bas niveau. Exclusion par <code>tfsec:ignore</code>

Tableau 33. – Constats tfsec sur le module Terraform et décisions associées.

5. 4.3 Interprétation des décisions

- **Correction** : Le **finding** sur IMDSv2 est légitime et facile à corriger. Le patch a été appliqué immédiatement.
- **Risque accepté (chiffrement at-rest)** : Le volume racine EC2 n'est pas chiffré. Dans un contexte AWS Academy sans données sensibles, ce risque est acceptable. La directive `tfsec:ignore` inclut une justification explicite pour l'auditeur.
- **Risque accepté (description)** : Une description manquante sur un security group n'a pas d'impact fonctionnel ou sécuritaire. Accepté comme non-bloquant.

5. 4.4 Intégration CI/CD

Le scan tfsec est exécuté sur chaque push dans le dépôt GitHub. Si une règle de sévérité HIGH ou CRITICAL est détectée (et non explicitement ignorée), le pipeline échoue immédiatement. Cela garantit qu'aucune configuration dangereuse n'atteint le déploiement sans avoir été examinée.

5. 5 Vérification de l'infrastructure déployée

5. 5.1 Résolution DNS

Ce qui a été testé : Les enregistrements DNS pointent-ils vers la bonne adresse IP ?

Résultat : `dig webcyber.app` confirme que le domaine et son alias `www.webcyber.app` résolvent correctement vers l'Elastic IP de l'instance EC2.

5. 5.2 État des conteneurs

Ce qui a été testé : Les trois conteneurs sont-ils en cours d'exécution avec les bons ports exposés ?

Résultat : `docker compose ps` confirme que `nginx`, `app` et `db` sont en état Up. Nginx expose les ports 80 et 443 ; Flask et PostgreSQL sont confinés au réseau Docker interne (aucun port publié sur l'hôte).

5. 5.3 Connectivité applicative

Ce qui a été testé : L'application répond-elle correctement sur HTTP et HTTPS ?

Résultat :

- `curl -I http://webcyber.app` → 301 Moved Permanently
- `curl -I https://webcyber.app` → 200 OK (HTTP/2)

La redirection fonctionne, et le certificat Let's Encrypt est actif.

5. 6 Scan de ports avec Nmap

5. 6.1 Objectif et méthode

Ce qui a été testé : Quels ports TCP sont accessibles depuis l'extérieur ?

Pourquoi ce test ? Un port ouvert inutilement est une porte d'entrée potentielle pour un attaquant. L'objectif est de confirmer que la surface d'attaque réseau correspond exactement à la conception (chapitre 3).

Commande utilisée : `nmap -sV -p- webcyber.app`

5. 6.2 Résultats et interprétation

Port	État	Service,	Analyse et justification
22/tcp	open	OpenSSH 8.x	Attendu : administration. Accès limité par clé SSH (pas de mot de passe)
80/tcp	open	Nginx 1.25 (redirect)	Attendu : redirection 301 permanente vers HTTPS
443/tcp	open	Nginx 1.25 (SSL)	Attendu : trafic web chiffré
5000/tcp	filtered	–	Correct : Flask non exposé (conforme à la conception Docker)
5432/tcp	filtered	–	Correct : PostgreSQL non exposé (conforme à la conception Docker)
Autres ports	filtered	–	Correct : surface d'attaque minimale

Tableau 34. – Résultats du scan Nmap.

5. 6.3 Conclusion et recommandations

La surface d'attaque réseau est **conforme à la conception**. Aucun service interne n'est joignable depuis Internet.

Recommandations pour une production réelle :

- Restreindre la source du port 22 (SSH) à une plage d'IP de confiance plutôt que 0.0.0.0/0
- Masquer la version de Nginx avec `server_tokens off` (déjà appliqué)

5. 7 Scan applicatif Web avec Nikto

5. 7.1 Objectif et méthode

Ce qui a été testé : L'application web présente-t-elle des mauvaises configurations HTTP classiques ?

Pourquoi ce test ? De nombreuses attaques exploitent des en-têtes HTTP manquants, des fichiers sensibles exposés (.git, .env), ou des informations de version divulguées.

Outil utilisé : Nikto (scanner boîte noire)

5. 7.2 Résultats et interprétation

Constat	Sévérité	Interprétation et mitigation
HSTS présent (max-age=31536000)	Info	Attendu : protège contre le SSL stripping. Configuré dans Nginx
CSP default-src 'self' présent	Info	Attendu : limite les scripts non autorisés (protection XSS)
X-Frame-Options: DENY présent	Info	Attendu : protège contre le clickjacking
X-Content-Type-Options: nosniff présent	Info	Attendu : empêche le MIME sniffing
Aucun fichier sensible accessible	Info	Attendu : pas de .git/, .env, backup exposés
Version Nginx non divulguée	Info	Attendu : server_tokens off actif
Aucune injection détectée	Info	Attendu : SQLAlchemy (requêtes para- métrées) + Jinja2 (auto-escape)

Tableau 35. – Synthèse des résultats Nikto.

5. 7.3 Conclusion

Aucune vulnérabilité de sévérité **critique** ou **haute** n'a été remontée. Les en-têtes de sécurité sont tous présents et correctement configurés. Les chemins sensibles sont fermés, et la version du serveur n'est pas divulguée.

5. 8 Évaluation TLS

5. 8.1 Objectif et méthode

Ce qui a été testé : La configuration TLS est-elle robuste ?

Pourquoi ce test ? 80% des attaques sur applications web impliquent une couche TLS mal configurée (versions obsolètes, algorithmes faibles, certificats invalides).

Outils utilisés :

- **SSL Labs Server Test** (Qualys) : évaluation externe
- **testssl.sh** : analyse locale détaillée

5. 8.2 Résultats et interprétation

Critère	Résultat
Note SSL Labs	A (95/100)
Protocoles supportés	TLS 1.2 et 1.3 uniquement
TLS 1.0 / 1.1 / SSLv3	Désactivés (aucune version obsolète)
Suites de chiffrement	Profil Mozilla Intermediate (algorithmes forts uniquement)
Certificat	Let's Encrypt valide, chaîne complète fournie
Forward Secrecy	Oui (ECDHE - aucune clé statique)
HSTS	Présent (max-age=31536000, préchargé)
OCSP Stapling	Activé (réduction de la latence de révocation)
Vulnérabilités connues	Non détectées (Heartbleed, POODLE, ROBOT, BEAST)

Tableau 36. – Synthèse de l'évaluation TLS.

5. 8.3 Conclusion

La configuration TLS est **robuste** et obtient une note A sur SSL Labs. Seules les versions TLS 1.2 et 1.3 sont supportées ; les algorithmes faibles (RC4, 3DES, MD5) sont exclus ; le **Forward Secrecy** est garanti.

Amélioration possible : Atteindre A+ nécessite d'implémenter HSTS avec préchargement (déjà applicable en production réelle).

5. 9 Synthèse des audits

Domaine	Statut	Commentaire
Sécurité de l'IaC (tfsec)	OK	0 finding actif non traité ; 2 exclusions documentées avec justification
IMDSv2 forcé sur l'EC2	OK	http_tokens = "required" (corrigé après audit tfsec)
Surface réseau (Nmap)	OK	Seuls 22, 80, 443 ouverts ; Flask/PostgreSQL invisibles
En-têtes HTTP (Nikto)	OK	HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy présents
Cookies de session	OK	Attributs Secure, HttpOnly, SameSite=Lax configurés
Configuration TLS	OK	TLS 1.2/1.3 uniquement, Forward Secrecy, note A sur SSL Labs
Hachage des mots de passe	OK	PBKDF2-SHA256 avec sel automatique (werkzeug)
Protection CSRF	OK	Token Flask-WTF vérifié sur tous les formulaires POST
Protection injection SQL	OK	ORM SQLAlchemy (requêtes paramétrées - pas de concaténation)
Protection XSS	OK	Jinja2 auto-escape + CSP restrictif en place
Isolation par utilisateur	OK	Filtre user_id systématique sur 100% des requêtes notes
Secrets (clés API, mots de passe)	OK	.env hors Git ; géré via GitHub Secrets dans CI/CD
Pipeline CI/CD	OK	Gate tfsec bloquant en amont du build et du deploy

Tableau 37. – Tableau de bord de sécurité.

5. 10 Limites de la campagne

La présente campagne d'audit ne prétend pas être exhaustive. Plusieurs angles morts doivent être mentionnés :

- **Outils boîte noire** : Nmap et Nikto sont des scanners automatisés. Ils ne remplacent pas un test d'intrusion manuel approfondi (tests d'API, logique métier complexe, attaques multi-étapes).
- **Absence de SAST sur le code Python** : Aucun outil d'analyse statique (bandit, semgrep) n'a été exécuté sur le code de l'application Flask. Cette lacune constitue une perspective d'amélioration.
- **Absence de test de charge** : Les performances sous forte charge n'ont pas été validées. L'instance t2.micro limite intrinsèquement le débit.
- **Absence de test de fuite mémoire** : Aucun test de vieillissement n'a été conduit.

Dans le cadre d'un projet semestriel S4, ces limites sont acceptables. Pour une mise en production réelle, un audit complémentaire serait nécessaire.

5. 11 Conclusion du chapitre

La phase de tests confirme deux points essentiels :

1. **Fonctionnellement** : L'application répond aux 12 user stories définies dans le chapitre 2 (inscription, authentification, CRUD, archive, corbeille, recherche, isolation).
2. **Sécurité** : Les six couches de défense (chapitre 3) ont été validées :
 - Security Group : seuls 3 ports ouverts
 - Système hôte : clé SSH, mises à jour auto
 - Docker : isolation réseau, utilisateur non-root
 - Nginx : TLS A, en-têtes de sécurité
 - Flask : mots de passe hachés, CSRF, isolation user_id
 - PostgreSQL : non exposé, requêtes paramétrées

Les principaux objectifs de sécurité fixés en début de projet sont remplis : isolation réseau, HTTPS strict, isolation des données, hachage robuste des mots de passe, et en-têtes HTTP durcis.

Chapitre 6

6 CONCLUSION GÉNÉRALE

Ce projet WebCyber a permis de concevoir, de réaliser et de déployer une application web sécurisée et conteneurisée, tout en respectant des objectifs clairs de qualité, de reproductibilité et de sécurité.

6. 1 Bilan du projet

Les principaux objectifs ont été atteints :

- une application de prise de notes fonctionnelle, avec authentification, gestion CRUD, archive, corbeille et recherche;
- une architecture conteneurisée composée de Nginx, Flask et PostgreSQL orchestrés par Docker Compose;
- un déploiement sur AWS avec EC2, Elastic IP, Security Group et Route 53;
- une chaîne HTTPS opérationnelle via Let's Encrypt, avec renouvellement automatisé;
- une approche de sécurité en profondeur fondée sur six couches : réseau, hôte, isolation Docker, proxy/TLS, application et base de données;
- une campagne de validation couvrant les tests fonctionnels et l'audit de sécurité (Nmap, Nikto, SSL Labs/testssl.sh).

Ce rapport montre que la solution développée est cohérente avec le périmètre académique du PFE tout en conservant des pratiques proches de l'industrie.

6. 2 Apports méthodologiques

Le travail réalisé a apporté plusieurs acquis importants :

- maîtrise des mécanismes de conteneurisation et des flux de données entre services Docker;
- compréhension de la sécurisation d'un déploiement web en production : gestion des certificats, configuration de Nginx, règles de pare-feu, isolation des services;
- capacité à documenter une architecture technique complète, de la conception à l'implémentation et au test;
- adoption d'une démarche d'audit pragmatique, fondée sur des outils standard et sur des conclusions actionnables.

6. 2.1 Retour sur la démarche agile

L'approche agile adoptée a démontré sa pertinence pour un projet académique individuel :

- **Visibilité** : le découpage en sprints a permis de montrer un avancement concret à chaque revue avec l'encadrant.
- **Adaptabilité** : les retours réguliers ont permis de corriger rapidement les choix techniques (configuration Docker, sécurité).
- **Qualité** : l'intégration précoce de la sécurité (DevSecOps) a réduit les corrections coûteuses en fin de projet.
- **Limites** : le travail individuel a restreint certaines pratiques agiles (absence de pair programming, daily remplacés par journal de bord personnel).

6. 3 Limites et perspectives d'évolution

Bien que la solution soit robuste pour un usage pédagogique et de démonstration, plusieurs améliorations restent possibles :

- mise en place d'un pipeline CI/CD pour automatiser les tests et le déploiement;
- supervision et journalisation centralisées (Prometheus, Grafana, ELK/CloudWatch);
- audit de code statique (bandit, semgrep) et tests de sécurité automatisés plus poussés;
- renforcement de l'authentification par 2FA ou OAuth;
- migration vers une solution d'orchestration plus avancée (Kubernetes, ECS) pour améliorer la résilience et l'extensibilité;
- sauvegardes automatisées de la base PostgreSQL et procédure de restauration.

Ces pistes constituent des prolongements naturels pour transformer WebCyber en une plateforme encore plus professionnelle et adaptée à un contexte de production.

6. 4 Conclusion

WebCyber illustre qu'un projet de fin d'études peut être à la fois simple dans son périmètre fonctionnel et ambitieux dans son approche architecturale et de sécurité. La démarche adoptée, centrée sur la reproductibilité et la défense en profondeur, offre une base solide pour évoluer vers des déploiements plus complexes et des applications plus critiques.

Bibliographie

- [1] National Institute of Standards and Technology (NIST), « The NIST Definition of Cloud Computing », n° Special Publication 800-145. National Institute of Standards, Technology, 2011.
- [2] OWASP Foundation, « OWASP Top Ten 2021 ». [En ligne]. Disponible sur: <https://owasp.org/Top10/>
- [3] Pallets Projects, « Flask Documentation ». [En ligne]. Disponible sur: <https://flask.palletsprojects.com/>
- [4] Docker, Inc., « Docker Documentation ». [En ligne]. Disponible sur: <https://docs.docker.com/>
- [5] Electronic Frontier Foundation (EFF), « Certbot Documentation ». [En ligne]. Disponible sur: <https://certbot.eff.org/>
- [6] OWASP Foundation, « CI/CD Security Best Practices ». [En ligne]. Disponible sur: <https://owasp.org/www-project-ci-cd-security/>